

中山大學
SUN YAT-SEN UNIVERSITY



国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

CUDA共享内存与常量内存

任课教师：吴迪

Roadmap

- Introducing CUDA Shared Memory
- Checking the Data Layout of Shared Memory
- Reducing Global Memory Access
- Coalescing Global Memory Accesses
- Constant Memory
- The Warp Shuffle Instruction

参考阅读：Cheng-Ch5

1. Introducing CUDA Shared Memory

1.1 Shared Memory

CUDA Shared Memory

GPUs are equipped with two types of memory:

- **On-board memory**: e.g., global memory
- **On-chip memory:** e.g., shared memory

Shared memory is generally useful as:

- An **intra-block thread communication** channel
- A **program-managed cache** for global memory data
- **Scratch pad memory （高速暂存器）** for transforming data to improve global memory access patterns

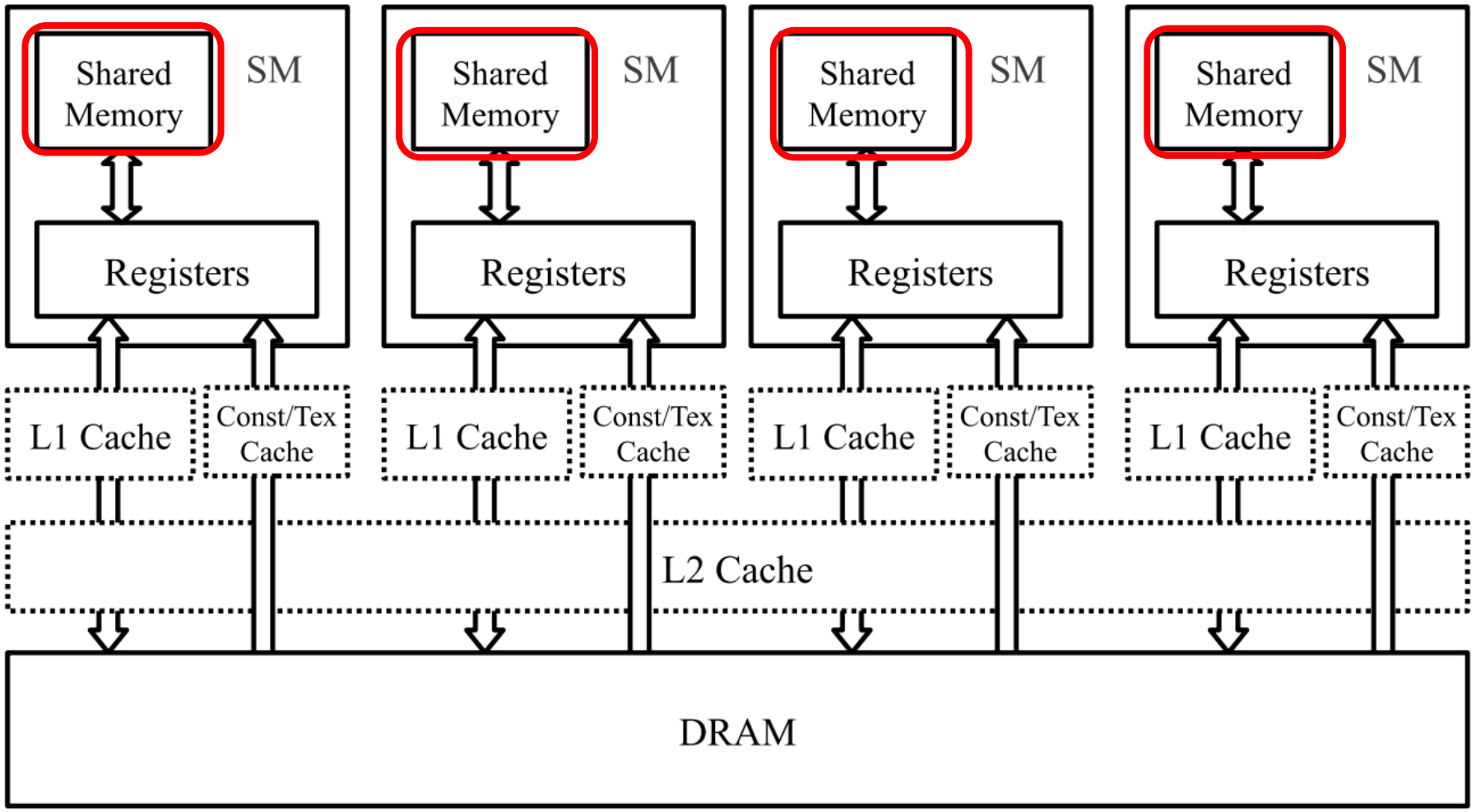
Shared Memory

- **Shared memory (SMEM)** is one of the key components of the GPU.

- **Each SM contains a small low-latency shared memory**
  - shared by all threads in the thread block on that SM.

- **Shared memory enables threads within the same thread block to cooperate**
  - facilitate reuse of on-chip data
  - reduce the global memory bandwidth needed by kernels.

Shared Memory



Shared Memory

- A fixed amount of shared memory is allocated to each thread block when it starts executing.
- This shared memory address space is shared by all threads in a thread block.
- Its contents have the same lifetime as the thread block.

Shared Memory

- Shared memory accesses are **issued per warp**.
- **Ideally**, each request to access shared memory by a warp is serviced in **one transaction**.
- **In the worst case,** each request to shared memory is executed sequentially in **32 unique transactions**.
- If multiple threads access **the same word** in shared memory, one thread fetches the word, and sends it to the other threads **via multicast**.

Shared Memory

- Shared memory is **partitioned** among **all resident thread blocks** on an SM

- Shared memory is a **critical resource** that limits device **parallelism**.

- The **more** shared memory used by a kernel, the **fewer** possible **concurrently active** thread blocks.

Program-managed Cache

- Shared memory is a **program-managed cache**.

- Programmer has full control over when data is moved into shared memory, and when data is evicted.

- **Easier to optimize** your application code by providing more fine-grained control
  - over data placement and on-chip data movement.

1.2 Shared Memory Allocation

Shared Memory Allocation

- You can allocate shared memory variables either **statically** or **dynamically**.
  - CUDA supports declaration of 1D, 2D, and 3D shared memory arrays.

- **Statically** declared with the following qualifier:

```
__shared__
```

- **Dynamically** allocated at each kernel invocation

```
extern __shared__ int tile[];
kernel<<<grid, block, isize*sizeof(int)>>>(...)
```

specifying the desired size in bytes as a third argument

Shared Memory Allocation

- Shared memory can be declared as either **local** to a CUDA kernel or **globally** in a CUDA source code file.

```
__shared__ float tile[size_y][size_x];
```

- If declared inside a kernel function, the scope of this variable is local to the kernel.

- If declared outside of any kernels in a file, the scope of this variable is global to all kernels.

1.3 Shared Memory Banks and Access Mode

Memory Banks

- Shared memory is divided into **32 equally-sized banks**
  - Banks can be accessed simultaneously.
  - There are 32 banks because there are 32 threads in a warp.
  - Addresses of shared memory are mapped to different banks in different patterns

- If a load or store issued by a warp access **<=1 memory location per bank**, serviced by **one memory transaction**.
  - Otherwise, serviced by multiple memory transactions, thereby decreasing memory bandwidth utilization.

Bank Conflict

- When multiple addresses in a request fall into the same memory bank, a bank conflict occurs
  - causing the request to be replayed.

- The hardware splits a request with a bank conflict into as many separate conflict-free transactions as necessary

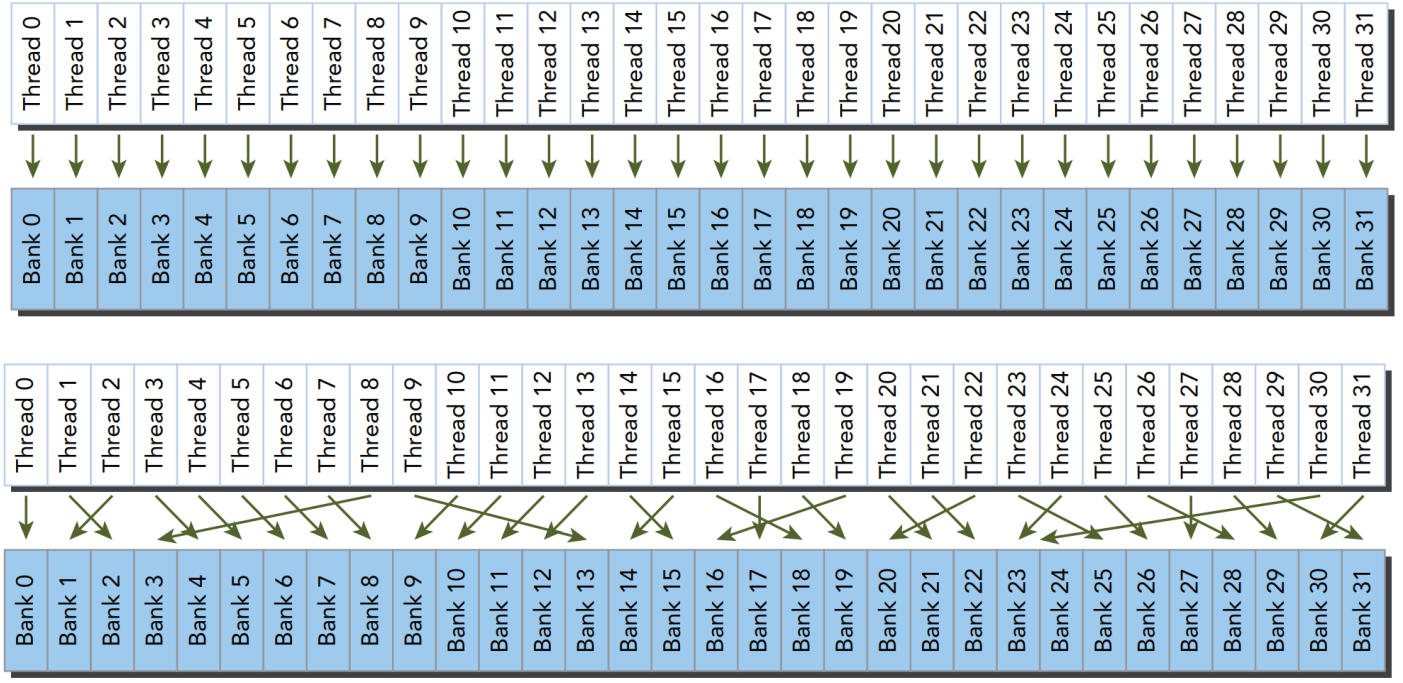
Bank Conflict

Three typical situations occur when a request to shared memory is issued by a warp:

- **Parallel access**: multiple addresses accessed across multiple banks
- **Serial access**: multiple addresses accessed within the same bank
- **Broadcast access**: a single address read in a single bank

Parallel Access

- Parallel access is the **most common** pattern:
  - multiple addresses accessed by a warp that fall into multiple banks.
- Optimally, a conflict-free shared memory access is performed when every address is in a separate bank.



Serial Access

- Serial access is the **worst** pattern
- When multiple addresses fall into the **same bank**, the request must be **serialized**.
- If all 32 threads in a warp access different memory locations in a single bank, 32 memory transactions will be required

Broadcast Access

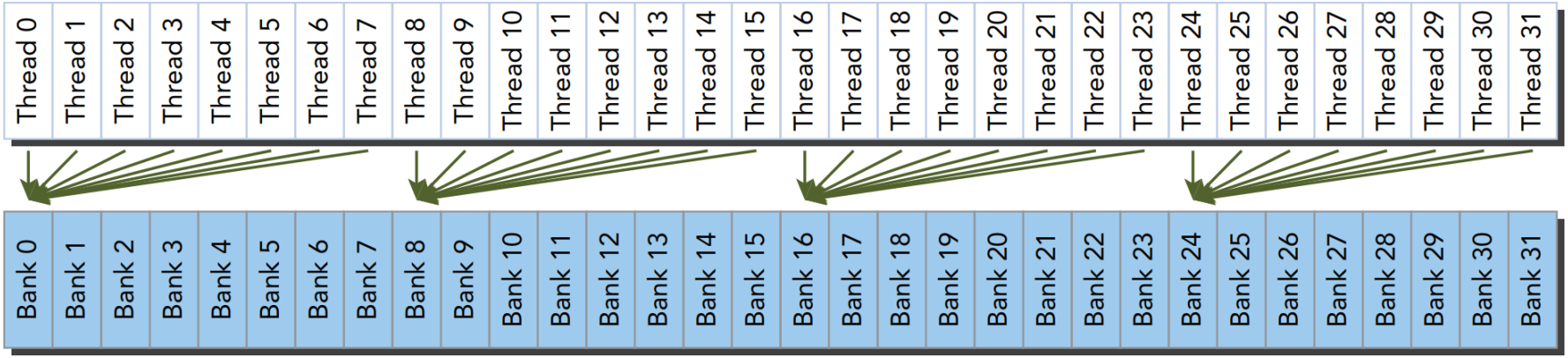
- All threads in a warp read the **same address** within a single bank.

- **One memory transaction** is executed, and the accessed word is broadcast to all requesting threads.

- While only a single memory transaction is required for a broadcast access, bandwidth utilization is poor
  - because only a small number of bytes are read.

Irregular Access Pattern

- Several threads access the same bank.



- There are two possible behaviors for such a request:
  - **Conflict-free broadcast access**: if threads access the same address within a bank
  - **Bank conflict access:** if threads access different addresses within a bank

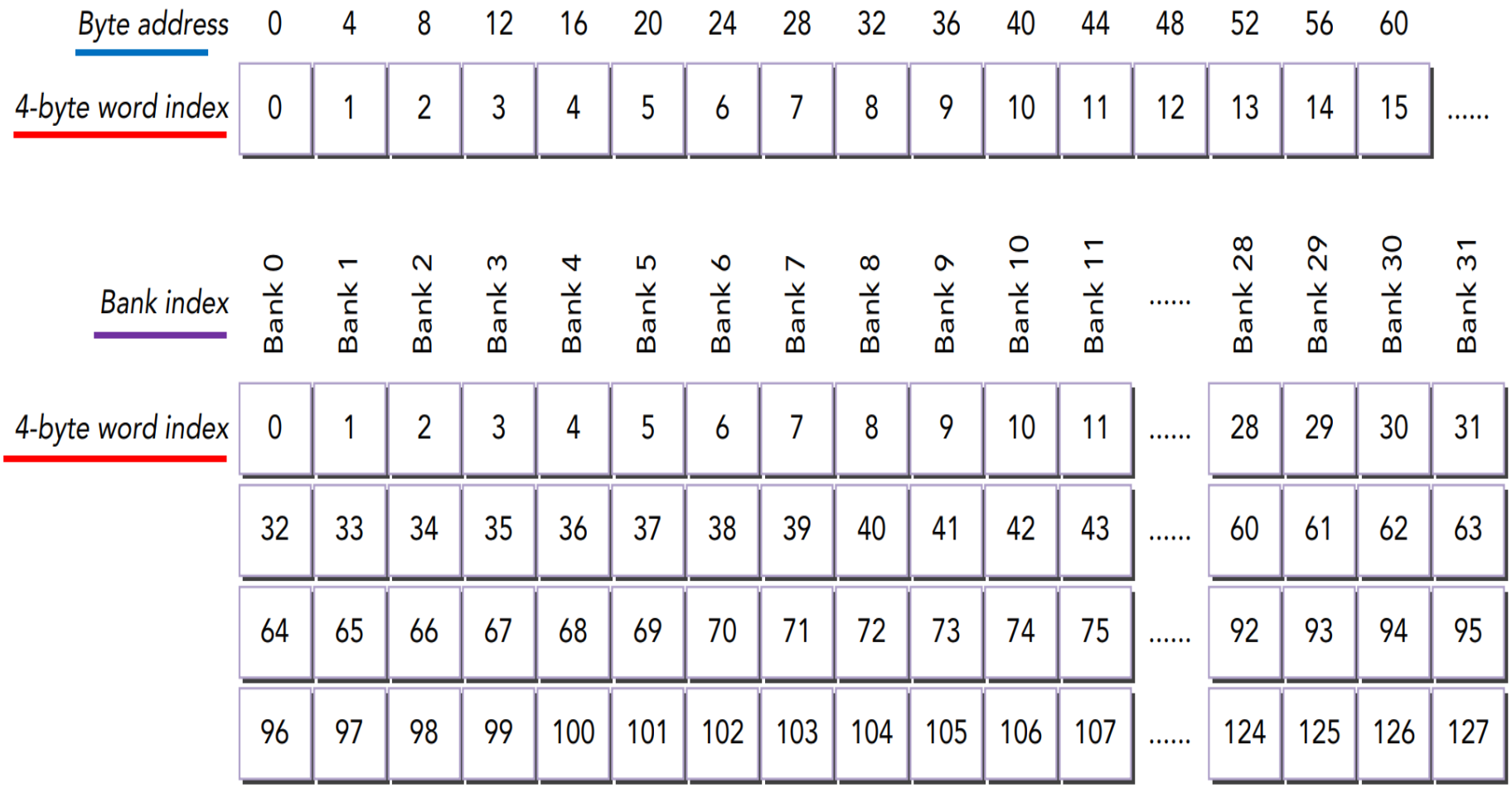
Access Mode

- Two different bank widths:
  - **4 bytes (32-bits)** for devices of compute capability 2.x
  - **8 bytes (64-bits)** for devices of compute capability 3.x

- For a Fermi device, the bank width is **32-bit**s and there are **32 banks**.
  - Each bank has a bandwidth of 32 bits per two clock cycles.
  - Successive 32-bit words map to successive banks.
  - The mapping from shared memory address to bank index can be calculated as follows:

$$bank\ index = (byte\ address \div 4\ bytes/bank)\ \%\ 32\ banks$$

Mapping to bank index



Kepler devices

- For Kepler devices, shared memory has 32 banks with the following two address modes:
  - 64-bit mode
  - 32-bit mode

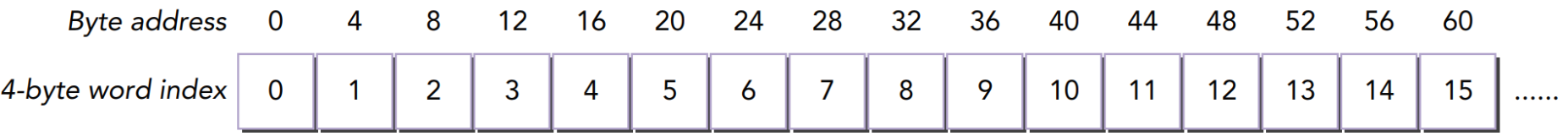
- In 64-bit mode, successive 64-bit words map to successive banks.
  - Each bank has a bandwidth of 64 bits per clock cycle.

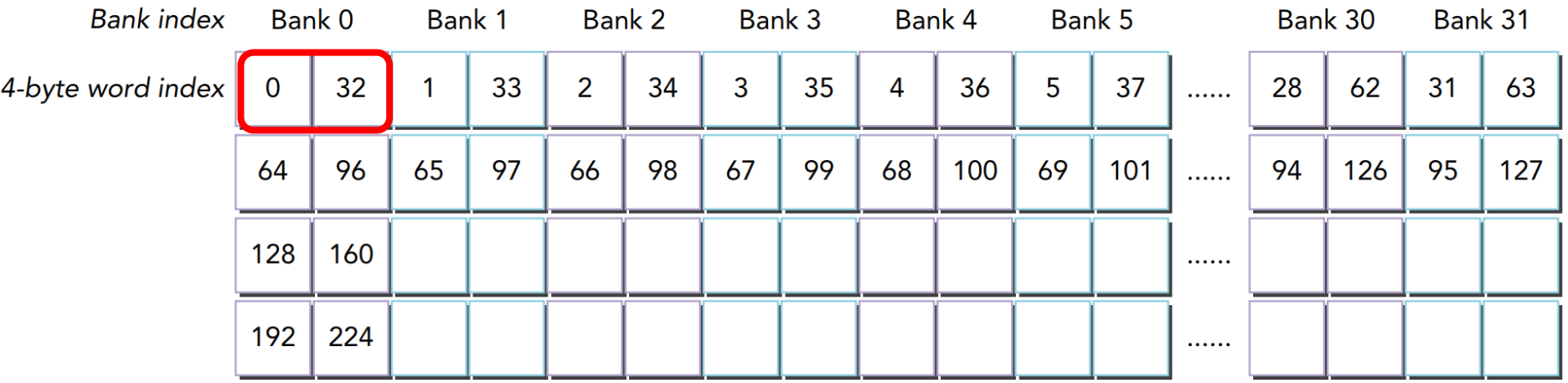
- The mapping from shared memory address to bank index can be calculated as follows:

$$bank\ index = (byte\ address \div 8\ bytes/bank)\ \%\ 32\ banks$$

Kepler devices (32-bit mode)

In 32-bit mode, successive 32-bit words map to successive banks.

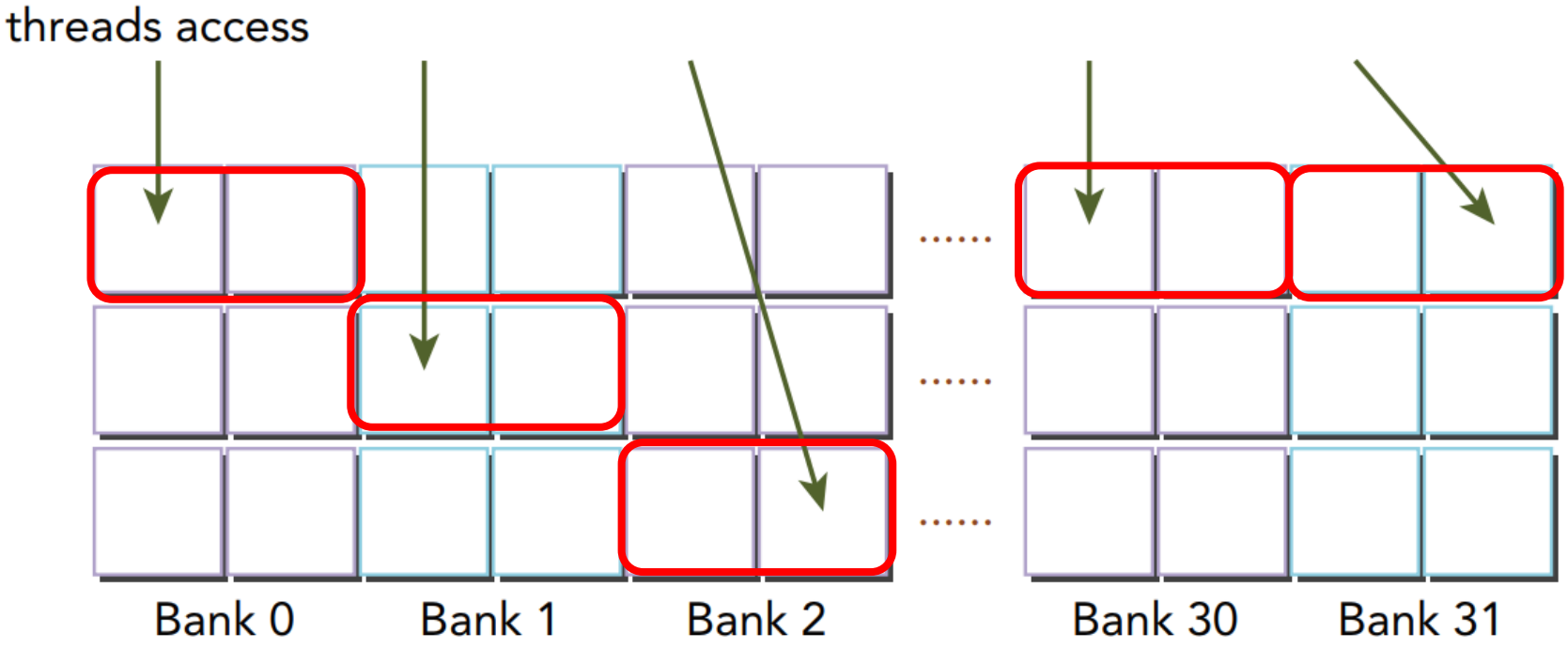




Because Kepler has a bandwidth of 64 bits per clock cycle, though word 0 and word 32 are both in bank 0, reading both in the same memory request would not imply a bank conflict.

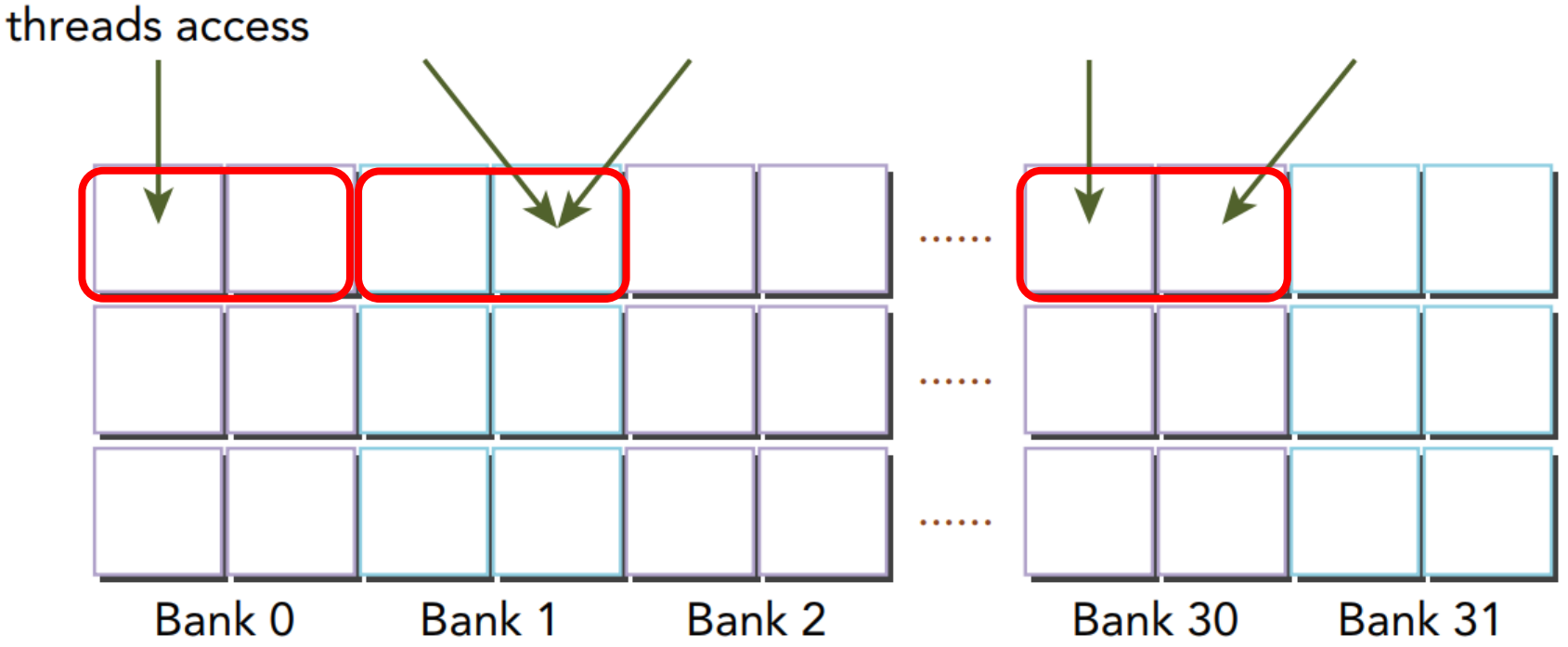
Kepler devices （64-bit mode）

- One case of conflict-free access in 64-bit mode, where each thread accesses different banks.



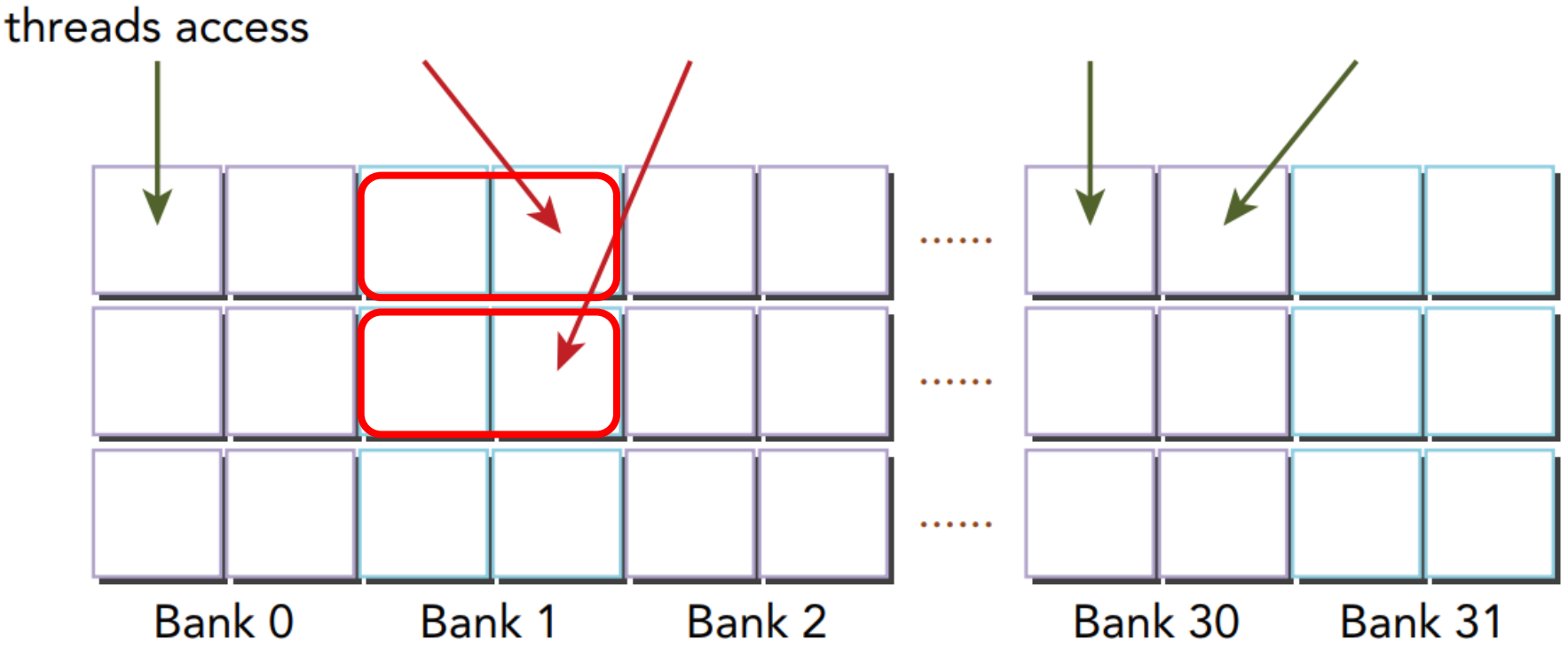
Kepler devices （64-bit mode）

- another case of conflict-free access in 64-bit mode
  - Where two threads access words in the same bank and within the same 8-byte word



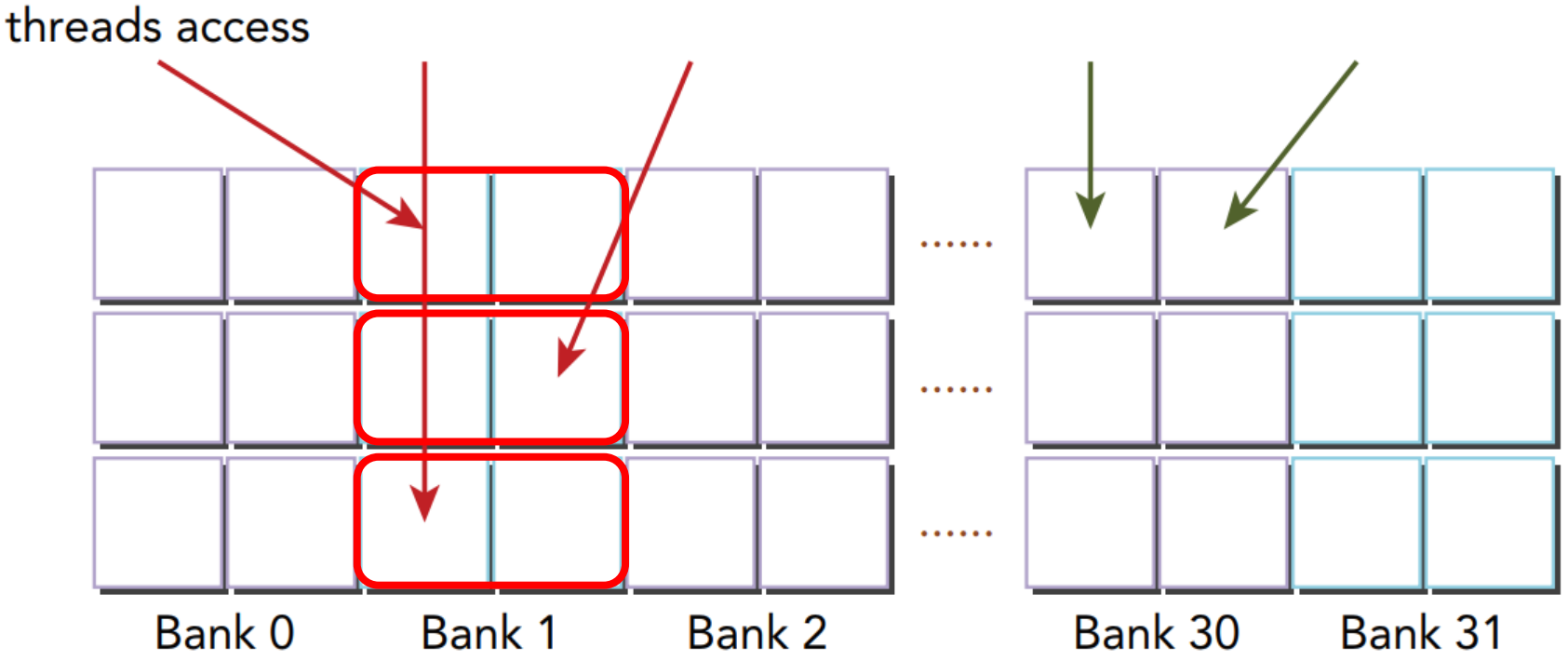
Kepler devices （64-bit mode）

- two-way **bank conflict** where two threads access the same bank, but the addresses fall in two different 8-byte words.



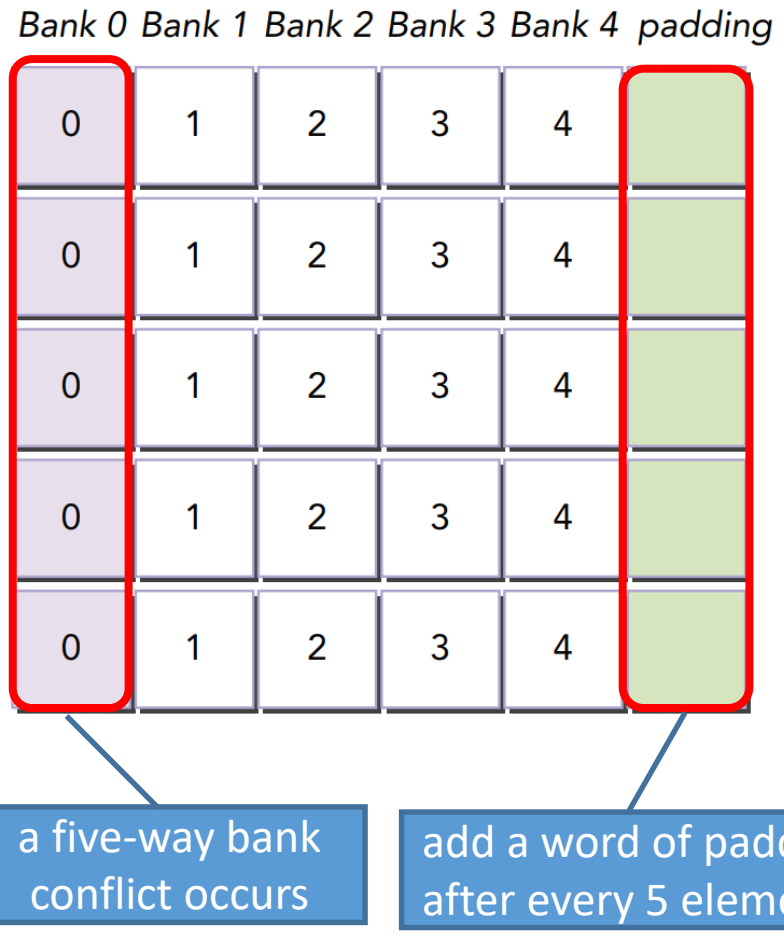
Kepler devices （64-bit mode）

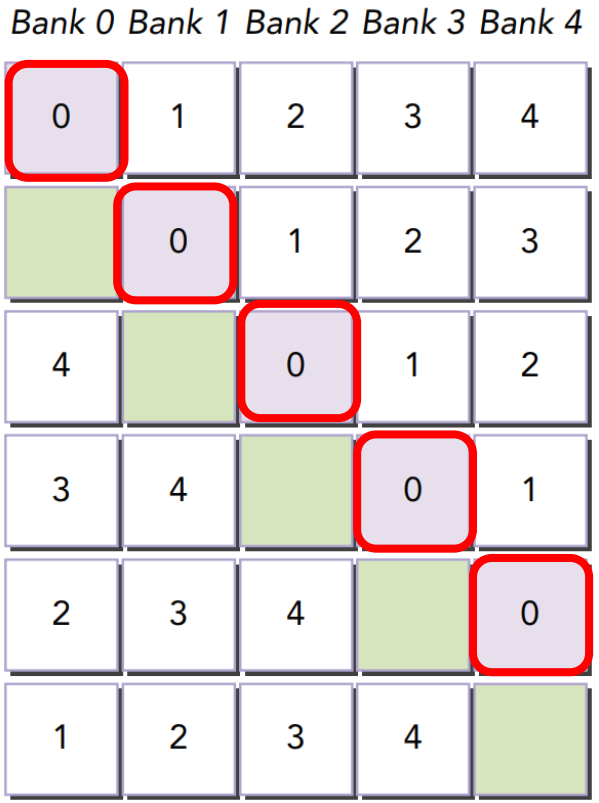
- three-way bank conflict, where three threads access the same bank and the addresses fall in 3 different 8-byte words



Memory Padding

- Memory padding is one way to avoid bank conflicts





- change the mapping from words to banks
- The words in bank 0 are spread across different banks

Memory Padding

- The padded memory is never used for data storage.
- Its sole function is to shift data elements
  - so that data that originally resided in the same bank are distributed among banks.
- The total amount of useful shared memory available to a thread block will decrease.
- Need to recalculate array indices after padding to access the correct data elements.

Access Mode Configuration

- The access mode can be queried using CUDA runtime API:

  cudaDeviceGetSharedMemConfig(cudaSharedMemConfig *pConfig);

- The returned bank configuration can be:

  cudaSharedMemBankSizeFourByte

  cudaSharedMemBankSizeEightByte

- Set a new bank size on devices with configurable shared memory banks:

  cudaDeviceSetSharedMemConfig(cudaSharedMemConfig config);

The supported bank configurations are:

- cudaSharedMemBankSizeDefault
- cudaSharedMemBankSizeFourByte
- cudaSharedMemBankSizeEightByte

1.4 Configuring the Amount of Shared Memory

Configuring Shared Memory

- Each SM has **64 KB** of on-chip memory.
- Shared memory and L1 cache **share** this hardware resource.
- CUDA provides **two methods** for configuring the size of L1 cache and shared memory:
  - Per-device configuration
  - Per-kernel configuration

Configuring Shared Memory

- You can configure the size of L1 cache and shared memory using API:

  cudaDeviceSetCacheConfig(cudaFuncCache cacheConfig);

- The supported cache configurations are:
  - cudaFuncCachePreferNone: no preference(default)
  - cudaFuncCachePreferShared: prefer 48KB shared memory and 16 KB L1 cache
  - cudaFuncCachePreferL1 : prefer 48KB L1 cache and 16 KB shared memory
  - cudaFuncCachePreferEqual: prefer 32KB L1 cache and 32 KB shared memory

Configuring Shared Memory

- Which mode is better depends on **how much you use shared memory** in your kernel.

- The typical cases are:
  - **Prefer more shared memory**: when a kernel uses more shared memory.
  - **Prefer more L1 cache:** when a kernel uses more registers.

Per-kernel Configuration

- A per-kernel configuration can override the device-wide setting, and can be set using API:

```
cudaFuncSetCacheConfig(const void* func,
    enum cudaFuncCacheca cheConfig);
```

The kernel function to apply this configuration to

- Even though L1 cache and shared memory are located in the same chip, they are different:
  - Shared memory is accessed through 32 banks, while L1 cache is accessed by cache line.
  - With shared memory, you have full control over what gets stored and where
  - while with L1 cache, data eviction is done by the hardware.

1.5 Synchronization

Synchronization

- Shared memory can be **simultaneously accessed** by multiple threads within a thread block.

- Cause **inter-thread conflicts** when the same shared memory location is modified by multiple threads without synchronization.

- There are two basic approaches to synchronization:
  - **Barriers**: At a barrier, all calling threads wait for all other calling threads to reach the barrier point.
  - **Memory fences:** At a memory fence, all calling threads stall until all modifications to memory are visible to all other calling threads.

Weakly-Ordered Memory Model

- Modern memory architectures have a relaxed memory model.
- Memory accesses are not necessarily executed in the order in which they appear in the program.
- CUDA adopts a weakly-ordered memory model to enable more aggressive compiler optimizations.
- To explicitly force a certain ordering for program correctness, memory fences and barriers must be inserted in application code.

Explicit Barrier

- In CUDA, it is only possible to perform a barrier among threads in the **same thread block**.

- You can specify a barrier point in a kernel by using:

```
void __syncthreads();
```

- **`__syncthreads`** ensures that all memory accesses made by these threads prior to the barrier point are **visible to all threads** in the same block.

Explicit Barrier

- Be particularly careful when using __syncthreads in **conditional code**.

- Only valid if a conditional is guaranteed to **evaluate identically** across the entire thread block.
  - Otherwise execution is likely to hang or produce unintended side effects.

```
if (threadID % 2 == 0) {
    __syncthreads();
} else {
    __syncthreads();
}
```

- Such code may cause threads in a block to wait indefinitely for each other
- Because all threads in a block never hit the same barrier point.

Memory Fence

- Memory fence ensures that any memory write before the fence is **visible to other threads after the fence**.

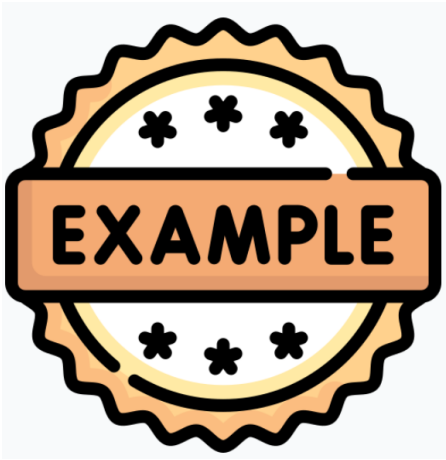
- There are three variants of memory fences depending on the desired scope:
  - **block-level**: `void __threadfence_block()`
  - **grid-level:** `void __threadfence()`
  - **system-level:** `void __threadfence_system()`

2. Checking the Data Layout of Shared Memory

How to use shared memory?

When designing kernels that use **shared memory,** your focus should be on the following **two concepts**:

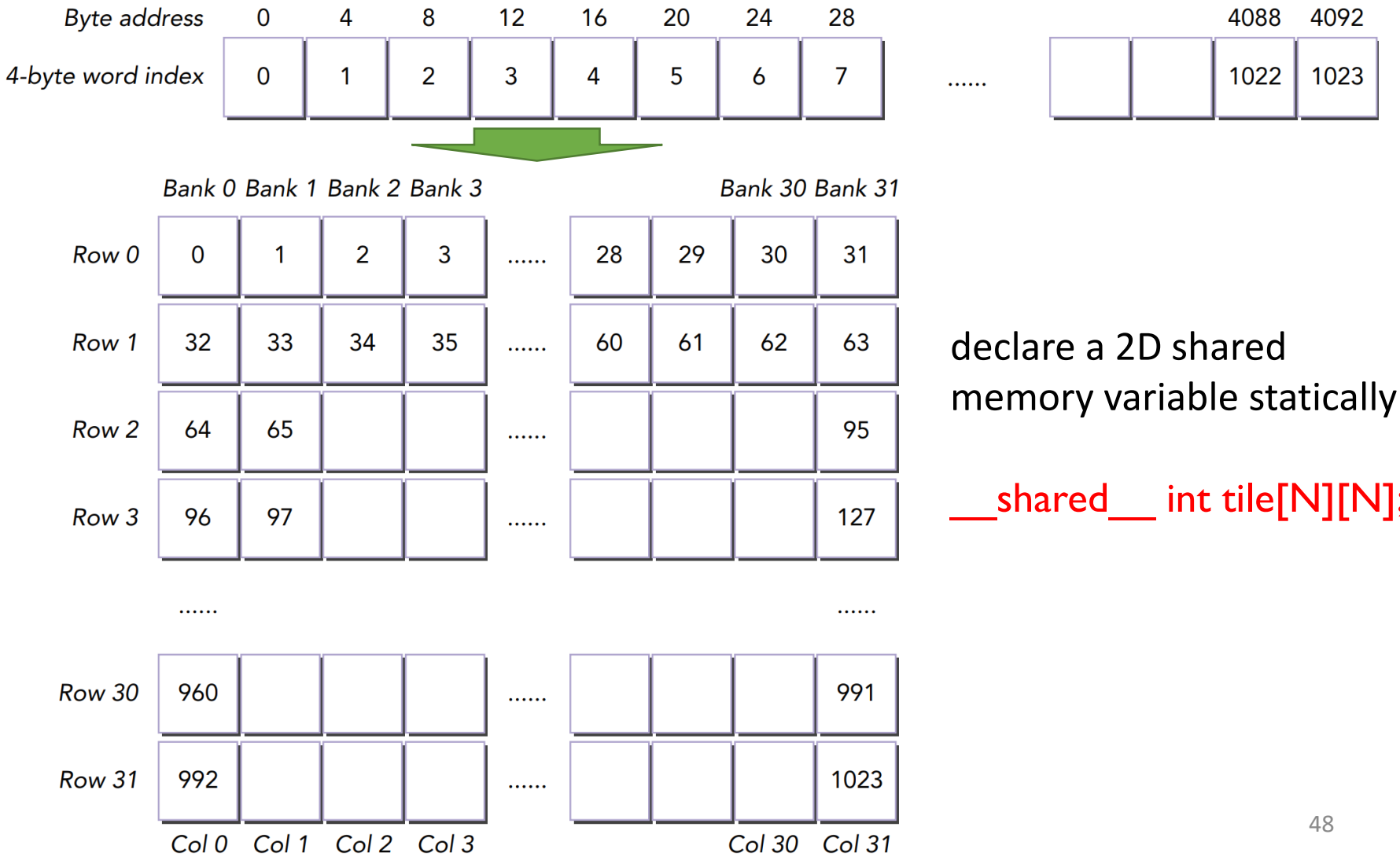
- Mapping **data elements** across **memory banks**
- Mapping from **thread index** to **shared memory offset**



2.1 Square Shared Memory

Square Shared Memory

- Use shared memory to cache 2d global data



declare a 2D shared memory variable statically

__shared__ int tile[N][N];

Square Shared Memory

- Which access method is likely to **perform better**?
  - `tile[threadIdx.y][threadIdx.x]`
  - `tile[threadIdx.x][threadIdx.y]`

- Recall that it is optimal to have **threads in the same warp** accessing **separate banks**.
  - It is best to have threads with consecutive values of threadIdx.x accessing consecutive locations in shared memory.

- From this, **the first access pattern** will exhibit **better performance** and fewer bank conflicts

Accessing Row-Major vs. Column-Major

- The kernel has two simple operations:
  - Write global thread indices to a 2D shared memory array in row-major order.
  - Read those values from shared memory in row-major order and store them to global memory

- 3 memory operations in the kernel:
  - One store operation on shared memory
  - One load operation on shared memory
  - One store operation on global memory

```
__global__ void setRowReadRow(int *out) {
    // static shared memory
    __shared__ int tile[BDIMY][BDIMX];

    // mapping from thread index to global memory index
    unsigned int idx = threadIdx.y * blockDim.x + threadIdx.x;

    // shared memory store operation
    tile[threadIdx.y][threadIdx.x] = idx;

    // wait for all threads to complete
    __syncthreads();

    // shared memory load operation
    out[idx] = tile[threadIdx.y][threadIdx.x] ;
}
```

declare a 2D shared memory array statically

One store operation on shared memory

One store operation on global memory

One load operation on shared memory

```
__global__ void setColReadCol(int *out) {
    // static shared memory
    __shared__ int tile[BDIMX][BDIMY];

    // mapping from thread index to global memory index
    unsigned int idx = threadIdx.y * blockDim.x + threadIdx.x;

    // shared memory store operation
    tile[threadIdx.x][threadIdx.y] = idx;

    // wait for all threads to complete
    __syncthreads();

    // shared memory load operation
    out[idx] = tile[threadIdx.x][threadIdx.y];
}
```



checkSmemSquare.cu

```
./smemSquare at device 0 of Tesla K40c with Bank Mode:4-byte
<<< grid (1,1) block (32,32)>>>
Time(%)      Time     Calls       Avg       Min       Max  Name
 13.25%  2.6880us         1  2.6880us  2.6880us  2.6880us  setColReadCol(int*)
 11.36%  2.3040us         1  2.3040us  2.3040us  2.3040us  setRowReadRow(int*)
```

accessing shared memory by row improved performance, as a result of neighboring threads referencing neighboring words.

```
Kernel:setColReadCol (int*)
    1   shared_load_transactions_per_request   16.000000
    1   shared_store_transactions_per_request  16.000000
Kernel:setRowReadRow(int*)
    1   shared_load_transactions_per_request   1.000000
    1   shared_store_transactions_per_request  1.000000
```

- requests from a warp in kernel setRowReadRow are serviced by 1 transaction
- the same requests in kernel setColReadCol are serviced by 16 transactions.

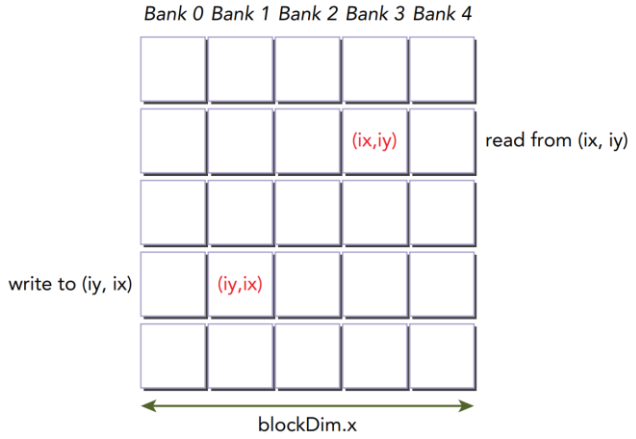
```
__global__ void setRowReadCol(int *out) {
    // static shared memory
    __shared__ int tile[BDIMY][BDIMX];

    // mapping from thread index to global memory index
    unsigned int idx = threadIdx.y * blockDim.x + threadIdx.x;

    // shared memory store operation
    tile[threadIdx.y][threadIdx.x] = idx;

    // wait for all threads to complete
    __syncthreads();

    // shared memory load operation
    out[idx] = tile[threadIdx.x][threadIdx.y];
}
```




```
Kernel:setRowReadCol (int*)
    1  shared_load_transactions_per_request  16.000000
    1  shared_store_transactions_per_request  1.000000
```

store operation is conflict-free, but load operation reports a 16-way conflict.

Dynamic Shared Memory

- implement these same kernels by declaring shared memory dynamically

```
__global__ void setRowReadColDyn(int *out) {
    // dynamic shared memory
    extern   __shared__  int tile[];

    // mapping from thread index to global memory index
    unsigned int row_idx = threadIdx.y * blockDim.x + threadIdx.x;
    unsigned int col_idx = threadIdx.x * blockDim.y + threadIdx.y;

    // shared memory store operation
    tile[row_idx] = row_idx;

    // wait for all threads to complete
    __syncthreads();

    // shared memory load operation
    out[row_idx] = tile[col_idx];
}
```

1D row-major memory offset calculated from 2D thread indices

1D column-major memory offset calculated from 2D thread indices

Dynamic Shared Memory

- shared memory size must be specified when launching the kernel

```
setRowReadColDyn<<<grid, block, BDIMX * BDIMY * sizeof(int)>>>(d_C);
```

- nvprof results:

```
Kernel: setRowReadColDyn(int*)
     1   shared_load_transactions_per_request   16.000000
     1   shared_store_transactions_per_request   1.000000
```

- The results are identical to the previous setRowReadCol,
- but use dynamically declared shared memory with 1D array index calculation instead.

Padding Statically Declared Shared Memory

- Simply add a column to 2D shared memory allocation:

__shared__ int tile[BDIMY][BDIMX+1];

```
__global__ void setRowReadColPad(int *out) {
    // static shared memory
    __shared__ int tile[BDIMY][BDIMX+IPAD];

    // mapping from thread index to global memory offset
    unsigned int idx = threadIdx.y * blockDim.x + threadIdx.x;

    // shared memory store operation
    tile[threadIdx.y][threadIdx.x] = idx;

    // wait for all threads to complete
    __syncthreads();

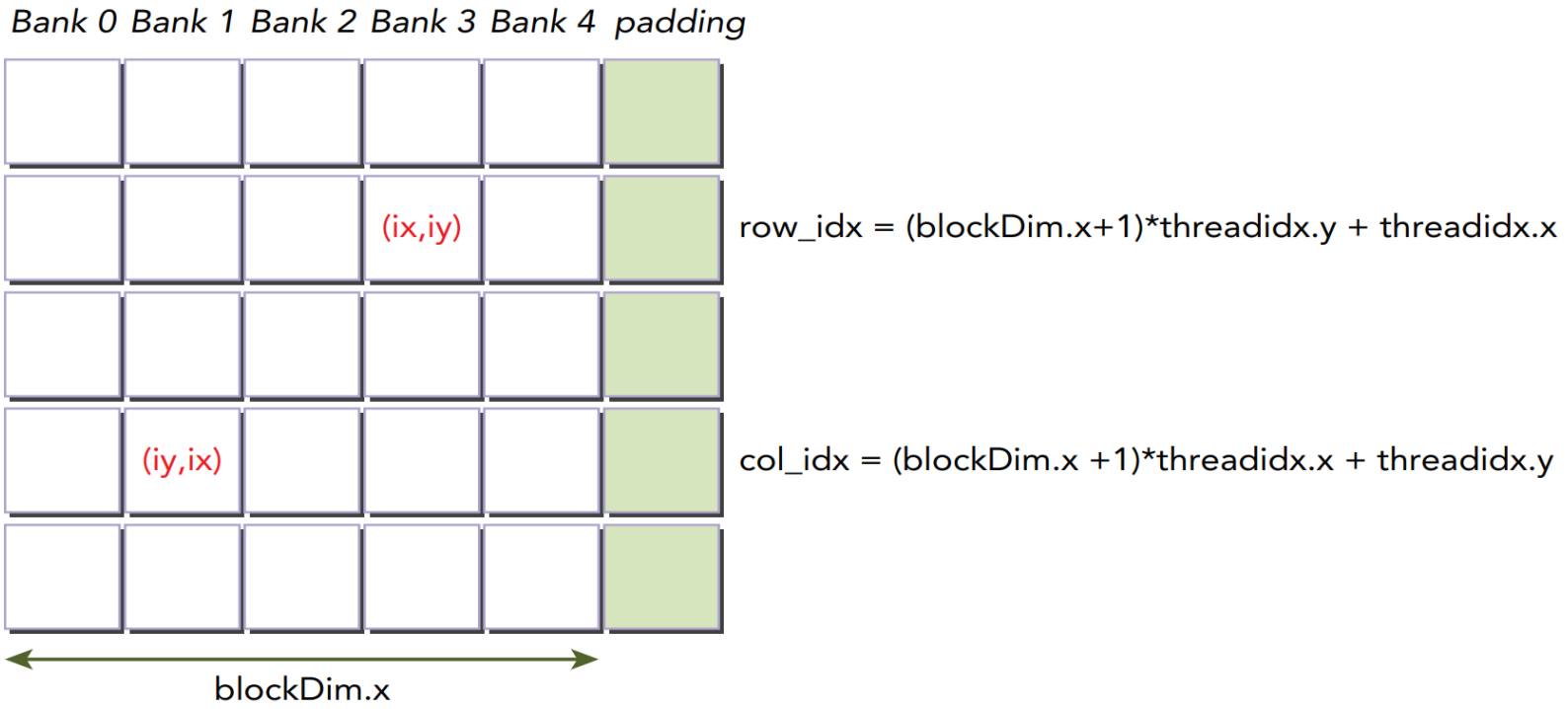
    // shared memory load operation
    out[idx] = tile[threadIdx.x][threadIdx.y];
}
```

```
Kernel: setRowReadColPad(int*)
    1  shared_load_transactions_per_request  1.000000
    1  shared_store_transactions_per_request 1.000000
```

Padding Dynamically Declared Shared Mem.

- Padding a dynamically declared shared memory array is more complex.
- Skip one padded memory space for each row when performing index conversion from 2D to 1D:

```
unsigned int row_idx = threadIdx.y * (blockDim.x + 1) + threadIdx.x;
unsigned int col_idx = threadIdx.x * (blockDim.x + 1) + threadIdx.y;
```



Padding Dynamically Declared Shared Mem.

```
setRowReadColDynPad<<<grid, block, (BDIMX + 1) * BDIMY * sizeof(int)>>>(d_C);
```

```
__global__ void setRowReadColDynPad(int *out) {
   // dynamic shared memory
   extern  __shared__ int tile[];

   // mapping from thread index to global memory index
   unsigned int row_idx = threadIdx.y * (blockDim.x + IPAD) + threadIdx.x;
   unsigned int col_idx = threadIdx.x * (blockDim.x + IPAD) + threadIdx.y;

   unsigned int g_idx = threadIdx.y * blockDim.x + threadIdx.x;

   // shared memory store operation
   tile[row_idx] = g_idx;

   // wait for all threads to complete
   __syncthreads();

   // shared memory load operation
   out[g_idx] = tile[col_idx];
}
```

Performance Comparison

- Comparing Performance of different kernels

```
$ nvprof ./smemSquare
./smemSquare at device 0: Tesla K40c with Bank Mode:4-Byte
<<< grid (1,1) block (32,32)>>>
Time(%)      Time  Calls       Avg       Min       Max  Name
  5.32%  3.6160us      1  3.6160us  3.6160us  3.6160us  setColReadCol(int*)
  4.57%  3.1040us      1  3.1040us  3.1040us  3.1040us  setRowReadColDyn(int*)
  4.24%  2.8800us      1  2.8800us  2.8800us  2.8800us  setColReadRow(int*)
  3.81%  2.5920us      1  2.5920us  2.5920us  2.5920us  setRowReadCol(int*)
  3.20%  2.1760us      1  2.1760us  2.1760us  2.1760us  setRowReadColDynPad(int*)
  3.15%  2.1440us      1  2.1440us  2.1440us  2.1440us  setRowReadRow(int*)
  3.15%  2.1440us      1  2.1440us  2.1440us  2.1440us  setRowReadColPad(int*)
```

```
$./smemSquare 1
./smemSquare at device 0: Tesla K40c with Bank Mode:4-Byte <<< grid (1,1) block
(4,4)>>>
set col read col          : 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
set row read row          : 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
set col read row          : 0  4  8 12  1  5  9 13  2  6 10 14  3  7 11 15
set row read col          : 0  4  8 12  1  5  9 13  2  6 10 14  3  7 11 15
set row read col  Dynamic : 0  4  8 12  1  5  9 13  2  6 10 14  3  7 11 15
set row read col  Padding : 0  4  8 12  1  5  9 13  2  6 10 14  3  7 11 15
set row read col  Dyn Pad : 0  4  8 12  1  5  9 13  2  6 10 14  3  7 11 15
```

3. Reducing Global Memory Access

3.1 Parallel Reduction with Shared Memory

Parallel Reduction with Shared Memory

- Recall reduceInteger.cu in Chapter 3
- The new kernel is nearly identical to the original kernel, but augments global memory operations with shared memory
- A shared memory array **`smem`** is used, which has the same dimensions as each thread block:

```
__shared__ int smem[DIM];
```

- In-place reduction is performed using shared memory (smem), rather than global memory (idata)

```
__global__ void reduceSmem(int *g_idata, int *g_odata, unsigned int n) {
    __shared__ int smem[DIM];

    // set thread ID
    unsigned int tid = threadIdx.x;

    // boundary check
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x;

    // set to smem by each threads
    smem[tid] = idata[tid];
    syncthreads();
```



reduceInteger.cu

```
    // in-place reduction in shared memory
    if (blockDim.x >= 1024 && tid < 512) smem[tid] += smem[tid + 512];
    __syncthreads();

    if (blockDim.x >= 512 && tid < 256) smem[tid] += smem[tid + 256];
    __syncthreads();

    if (blockDim.x >= 256 && tid < 128) smem[tid] += smem[tid + 128];
    __syncthreads();

    if (blockDim.x >= 128 && tid < 64)  smem[tid] += smem[tid + 64];
    __syncthreads();

    // unrolling warp
    if (tid < 32) {
       volatile int *vsmem = smem;
       vsmem[tid] += vsmem[tid + 32];
       vsmem[tid] += vsmem[tid + 16];
       vsmem[tid] += vsmem[tid +  8];
       vsmem[tid] += vsmem[tid +  4];
       vsmem[tid] += vsmem[tid +  2];
       vsmem[tid] += vsmem[tid +  1];
    }
    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = smem[0];
}
```



reduceInteger.cu

Performance Comparison

```
reduce at device 0: Tesla K40c with array size 16777216  grid 131072 block 128
Time(%)      Time     Calls       Avg       Min       Max  Name
  2.01%  2.1206ms         1  2.1206ms  2.1206ms  2.1206ms  reduceGmem()
  1.10%  1.1536ms         1  1.1536ms  1.1536ms  1.1536ms  reduceSmem()
```

The kernel using shared memory is **1.84 times faster** than the kernel using only global memory.

```
Device "Tesla K40c (0)"
        Kernel: reduceSmem(int*, int*, unsigned int)
          1                Global Load Transactions     524288
          1                Global Store Transactions    131072
        Kernel: reduceGmem(int*, int*, unsigned int)
          1                Global Load Transactions    2883584
          1                Global Store Transactions   1179648
```

Using shared memory reduced global memory access by a significant amount.

3.2 Parallel Reduction with Unrolling

Parallel Reduction with Unrolling

- Each thread block handles 1 data block → **4 data blocks**

- Benefits with unrolling:
  - Increased global memory throughput by exposing more parallel I/O per thread
  - Reduce global memory store transactions to ¼
  - Overall kernel performance improvement

```
__global__ void reduceSmemUnroll(int *g_idata, int *g_odata, unsigned int n) {
   // static shared memory
   __shared__ int smem[DIM];

   // set thread ID
   unsigned int tid = threadIdx.x;

   // global index, 4 blocks of input data processed at a time
   unsigned int idx = blockIdx.x * blockDim.x * 4 + threadIdx.x;

   // unrolling 4 blocks
   int tmpSum = 0;

   // boundary check
   if (idx + 3 * blockDim.x <= n) {
      int a1 = g_idata[idx];
      int a2 = g_idata[idx + blockDim.x];
      int a3 = g_idata[idx + 2 * blockDim.x];
      int a4 = g_idata[idx + 3 * blockDim.x];
      tmpSum = a1 + a2 + a3 + a4;
   }

   smem[tid] = tmpSum;
```

recalculating the offset in the global input data

From this new offset, each thread reads four data elements and then adds them to tmpSum.

```
    __syncthreads();

    // in-place reduction in shared memory
    if (blockDim.x >= 1024 && tid < 512) smem[tid] += smem[tid + 512];
    __syncthreads();

    if (blockDim.x >= 512 && tid < 256)  smem[tid] += smem[tid + 256];
    __syncthreads();

    if (blockDim.x >= 256 && tid < 128)  smem[tid] += smem[tid + 128];
    __syncthreads();

    if (blockDim.x >= 128 && tid < 64)   smem[tid] += smem[tid + 64];
    __syncthreads();

    // unrolling warp
    if (tid < 32) {
       volatile int *vsmem = smem;
       vsmem[tid] += vsmem[tid + 32];
       vsmem[tid] += vsmem[tid + 16];
       vsmem[tid] += vsmem[tid +  8];
       vsmem[tid] += vsmem[tid +  4];
       vsmem[tid] += vsmem[tid +  2];
       vsmem[tid] += vsmem[tid +  1];
    }
    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = smem[0];
}
```

Performance Comparison

```
reduce at device 0: Tesla K40c with array size 16777216  grid 131072 block 128
Time(%)      Time     Calls       Avg       Min       Max  Name
  1.10%  1.1536ms         1  1.1536ms  1.1536ms  1.1536ms  reduceSmem()
  0.40%  418.27us         1  418.27us  418.27us  418.27us  reduceSmemUnroll()
```

reduceSmemUnroll is **2.76 times** faster than reduceSmem

```
Kernel: reduceSmem(int*, int*, unsigned int)
          1                Global Load Transactions     524288
          1                Global Store Transactions    131072
Kernel: reduceSmemUnroll(int*, int*, unsigned int)
          1                Global Load Transactions     524288
          1                Global Store Transactions     32768
```

With unrolling, # of global memory loads is unchanged,
but # of global memory store is reduced to **one-fourth**.

3.3 Parallel Reduction with Dynamic Shared Memory

Parallel Reduction with Dynamic Shared Memory

- Replace the static shared memory declaration with:

```
extern __shared__ int smem[];
```

- When launching the kernel, the amount of shared memory to dynamically allocate must be specified:

```
reduceSmemUnrollDyn<<<grid.x / 4, block,
DIM * sizeof(int)>>>(d_idata, d_odata, size);
```

- No significant difference between kernels using dynamically allocated shared memory and statically allocated shared memory.

3.4 Effective Bandwidth

Effective Bandwidth

- Effective bandwidth is the amount of I/O performed (in bytes) over the kernel's complete execution time.

effective bandwidth = (bytes read + bytes written) ÷ (time elapsed × 10^9) GB/s

KERNELS	ELAPSED TIME (MS)	READ DATA ELEMENTS	WRITE DATA ELEMENTS	TOTAL BYTES	BANDWIDTH (GB/S)
`reduceGmem`	2.1357	16777216	131072	67633152	31.67
`reduceSmem`	1.1206	16777216	131072	67633152	60.35
`reduceSmemUnroll`	0.4171	16777216	32768	67239936	161.21
`reduceSmemUnrollDyn`	0.4169	16777216	32768	67239936	161.29

- Significant effective bandwidth improvement by unrolling blocks.
- Unrolling causes each thread to expose more requests in-flight at once, leading to higher saturation of the memory bus.

4. Coalescing Global Memory Accesses

Coalescing Global Memory Accesses

- Using shared memory can help **avoid non-coalesced global memory access**.

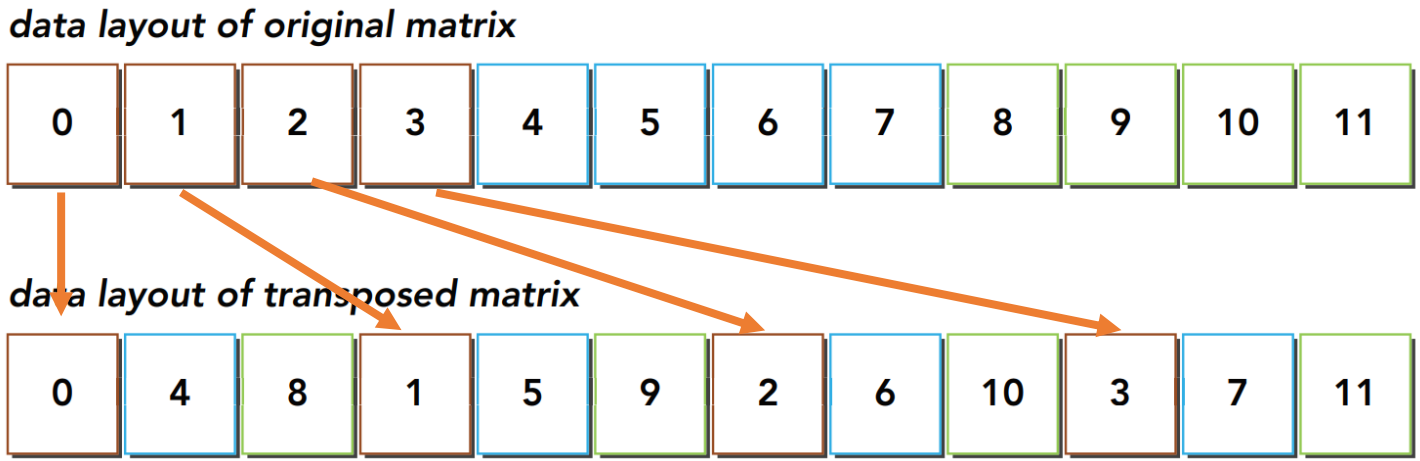
- **Matrix transpose** is a typical example:
  - Read is naturally coalesced, but write is accessed in stride.
  - Strided access is the worst access pattern for global memory
    - because it wastes bus bandwidth.

- **Improvement**:
  - perform transpose first in shared memory
  - then perform coalesced writes to global memory.

Recall: Transpose

```
for (int iy = 0; iy < ny; ++iy) {
   for (int ix = 0; ix < nx; ++ix) {
      out[ix*ny+iy] = in[iy*nx+ix];
   }
}
```

- **Reads**:
  - accessed by rows in the original matrix;
  - results in coalesced access.
- **Writes**:
  - accessed by columns in the transposed matrix;
  - results in strided access.



4.1 Baseline Transpose Kernel

Baseline Transpose Kernel

- **`naiveGmem:`** a **lower performance bound**
  - A naive implementation of matrix transpose that uses only global memory.
  - global memory read operation is coalesced within a warp
  - global memory write operation exhibits strided access between neighboring threads.

```
__global__ void naiveGmem(float *out, float *in, const int nx, const int ny) {
   // matrix coordinate (ix,iy)
   unsigned int ix = blockIdx.x * blockDim.x + threadIdx.x;
   unsigned int iy = blockIdx.y * blockDim.y + threadIdx.y;

   // transpose with boundary test
   if (ix < nx && iy < ny) {
      out[ix*ny+iy]= in[iy*nx+ix];
   }
}
```

Baseline Transpose Kernel

- **`copyGmem`:** an approximate **upper bound**:
  - read and write operations would be coalesced
  - but still performing the same amount of I/O

```
__global__ void copyGmem(float *out, float *in, const int nx, const int ny) {
   // matrix coordinate (ix,iy)
   unsigned int ix = blockIdx.x * blockDim.x + threadIdx.x;
   unsigned int iy = blockIdx.y * blockDim.y + threadIdx.y;

   // transpose with boundary test
   if (ix < nx && iy < ny) {
      out[iy * nx + ix] = in[iy * nx + ix];
   }
}
```



transposeRectangle.cu

Performance of Transpose Kernels

KERNELS	TESLA M2090 (ECC OFF)		TESLA K40C (ECC OFF)	
	ELAPSED TIME (MS)	BANDWIDTH (GB/S)	ELAPSED TIME (MS)	BANDWIDTH (GB/S)
copyGmem	1.048	128.07	0.758	177.15
naiveGmem	3.611	37.19	1.947	68.98

The naive kernel is nearly **three times slower** than the copy kernel.

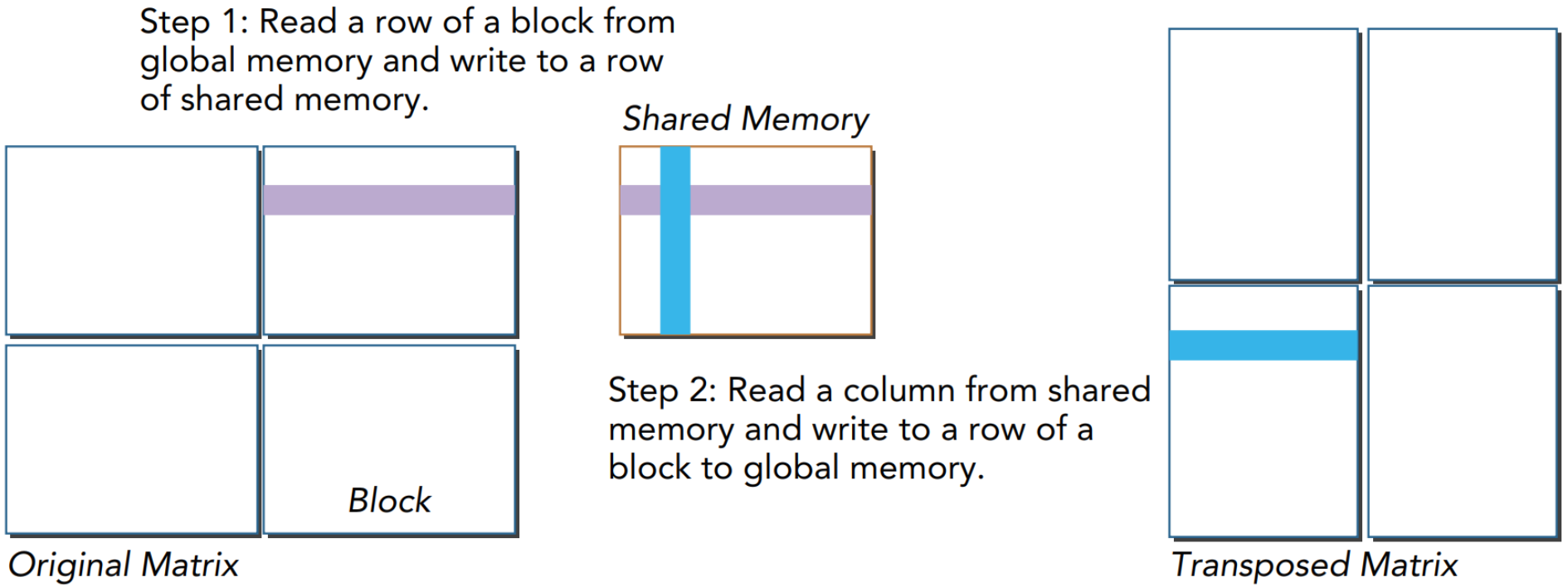
```
Device "Tesla K40c (0)"   Metrics     Transactions
Kernel:copyGmem(float*, float*, int, int)
  1    gld_transactions_per_request    1.000000
  1    gst_transactions_per_request    1.000000
Kernel:naiveGmem(float*, float*, int, int)
  1    gld_transactions_per_request    1.000000
  1    gst_transactions_per_request   32.000000
```

store requests to global memory are replayed **32 times** in naiveGmem.

4.2 Matrix Transpose with Shared Memory

Matrix Transpose with Shared Memory

- **2D shared memory** is used to cache data from original matrix
  - To avoid strided global memory access
- A **column read** from 2D shared memory can be transferred to a transposed matrix row stored in global memory.



Main Steps

- 1. A warp performs a coalesced read of a row from a block of the original matrix stored in global memory.
- 2. The warp then writes the data into shared memory using row-major ordering.
  - As a result, there are no bank conflicts for this write.
- 3. After all read/write operations in the thread block are synchronized, you have a 2D shared memory array filled with data from global memory
- 4. The warp reads a column from the 2D shared memory array.
  - Since the shared memory is not padded, bank conflicts occur.
- 5. The warp then performs a coalesced write of that data into a row of the transposed matrix stored in global memory

```
__global__ void transposeSmem(float *out, float *in, int nx, int ny) {
  // static shared memory
  __shared__ float tile[BDIMY][BDIMX];

  // coordinate in original matrix
  unsigned int ix,iy,ti,to;
  ix = blockIdx.x *blockDim.x + threadIdx.x;
  iy = blockIdx.y *blockDim.y + threadIdx.y;

  // linear global memory index for original matrix
  ti = iy*nx + ix;

  // thread index in transposed block
  unsigned int bidx,irow,icol;
  bidx = threadIdx.y*blockDim.x + threadIdx.x;
  irow = bidx/blockDim.y;
  icol = bidx%blockDim.y;

  // coordinate in transposed matrix
  ix = blockIdx.y * blockDim.y + icol;
  iy = blockIdx.x * blockDim.x  + irow;

  // linear global memory index for transposed matrix
  to = iy*ny + ix;
```

indices to the corresponding transposed block

The index into global memory used to store the transposed matrix

a warp of threads can then read contiguously from global memory and write to a row of the 2D shared memory array tile

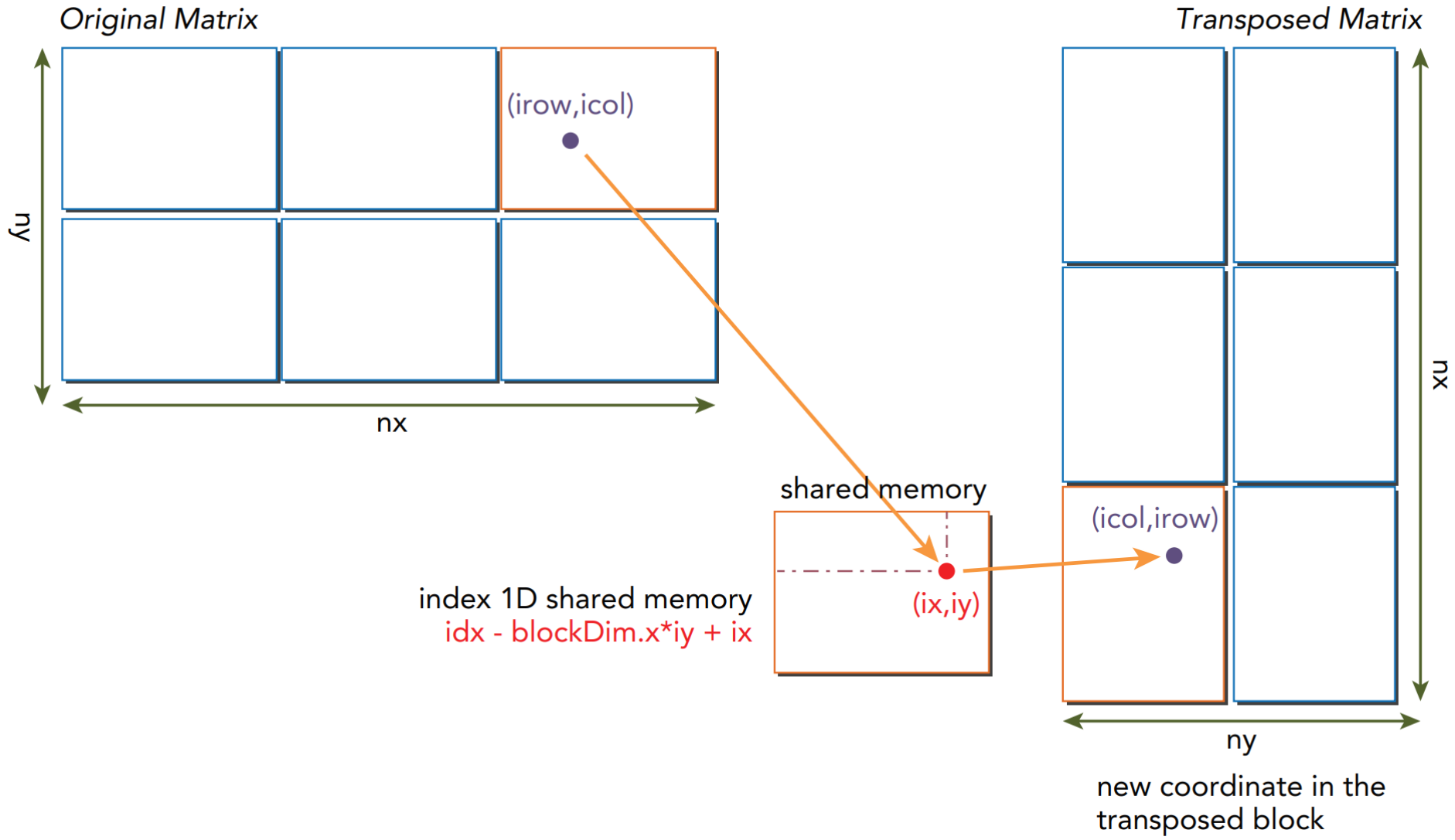
```
// transpose with boundary test
if (ix < nx && iy < ny)
{
    // load data from global memory to shared memory
    tile[threadIdx.y][threadIdx.x] = in[ti];

    // thread synchronization
    __syncthreads();

    // store data to global memory from shared memory
    out[to] = tile[icol][irow];
}
}
```

A warp of threads read a column from shared memory tile and write contiguously to global memory

Index Calculation



Performance of Transpose Kernels

KERNELS	TESLA M2090 (ECC OFF)		TESLA K40 (ECC OFF)	
	ELAPSED TIME (MS)	BANDWIDTH (GB/S)	ELAPSED TIME (MS)	BANDWIDTH (GB/S)
copyGmem	1.048	128.07	0.758	177.15
naiveGmem	3.611	37.19	1.947	68.98
transposeSmem	1.551	86.54	1.149	116.82

```
Device "Tesla K40c (0)"   Metrics     Transactions
Kernel: transposeSmem (float*, float*, int, int)
  1    gld_transactions_per_request  1.000000
  1    gst_transactions_per_request  2.000000
```

The replay of global memory stores is reduced from 32 to 2.

```
Device "Tesla K40c (0)"   Metrics     Transactions
Kernel: transposeSmem (float*, float*, int, int)
  1    shared_load_transactions_per_request   8.000000
  1    shared_store_transactions_per_request  1.000000
```

Reads from a column of the 2D shared memory array create bank conflicts

4.3 Matrix Transpose with Padded Shared Memory

Matrix Transpose with Padded Shared Memory

- By adding **column padding**, data elements in **the same column** can be **evenly distributed** among **shared memory banks**.
  - The number of columns that must be padded depends on device compute capability and thread block size.
  - Padding columns gives you additional speedup

```
__shared__ float tile[BDIMY][BDIMX + 2];
```

Matrix Transpose with Padded Shared Memory

KERNELS	TESLA M2090 (ECC OFF)		TESLA K40 (ECC OFF)	
	ELAPSED TIME (MS)	BANDWIDTH (GB/S)	ELAPSED TIME (MS)	BANDWIDTH (GB/S)
copyGmem	1.048	128.07	0.758	177.15
naiveGmem	3.611	37.19	1.947	68.98
transposeSmem	1.551	86.54	1.149	116.82
transposeSmemPad	1.416	94.79	1.102	121.83

```
Device "Tesla K40c (0)"   Metrics    Transactions
Kernel: transposeSmemPad (float*, float*, int, int)
  1    shared_load_transactions_per_request   1.000000
  1    shared_store_transactions_per_request  1.000000
```

Adding column padding to the shared memory array eliminated all bank conflicts.

4.4 Matrix Transpose with Unrolling

Matrix Transpose with Unrolling

- Unroll simultaneous processing of **two data blocks**:
  - Each thread transposes two data elements strided by one data block.

- The goal is to **improve device memory bandwidth utilization**
  - by creating more simultaneous in-flight loads and stores

```
__global__ void transposeSmemUnrollPad(float *out, float *in, const int nx,
      const int ny) {
   // static 1D shared memory with padding
   __shared__ float tile[BDIMY*(BDIMX*2+IPAD)];

   // coordinate in original matrix
   unsigned int ix = 2 * blockIdx.x * blockDim.x + threadIdx.x;
   unsigned int iy = blockIdx.y * blockDim.y + threadIdx.y;

   // linear global memory index for original matrix
   unsigned int ti = iy*nx + ix;

   // thread index in transposed block
   unsigned int bidx = threadIdx.y * blockDim.x + threadIdx.x;
   unsigned int irow = bidx / blockDim.y;
   unsigned int icol = bidx % blockDim.y;

   // coordinate in transposed matrix
   unsigned int ix2 = blockIdx.y * blockDim.y + icol;
   unsigned int iy2 = 2 * blockIdx.x * blockDim.x + irow;

   // linear global memory index for transposed matrix
   unsigned int to = iy2*ny + ix2;
```

```
    if (ix+blockDim.x < nx && iy < ny)
    {
        // load two rows from global memory to shared memory
        unsigned int row_idx = threadIdx.y * (blockDim.x * 2 + IPAD) + threadIdx.x;
        tile[row_idx]        = in[ti];
        tile[row_idx+BDIMX] = in[ti+BDIMX];

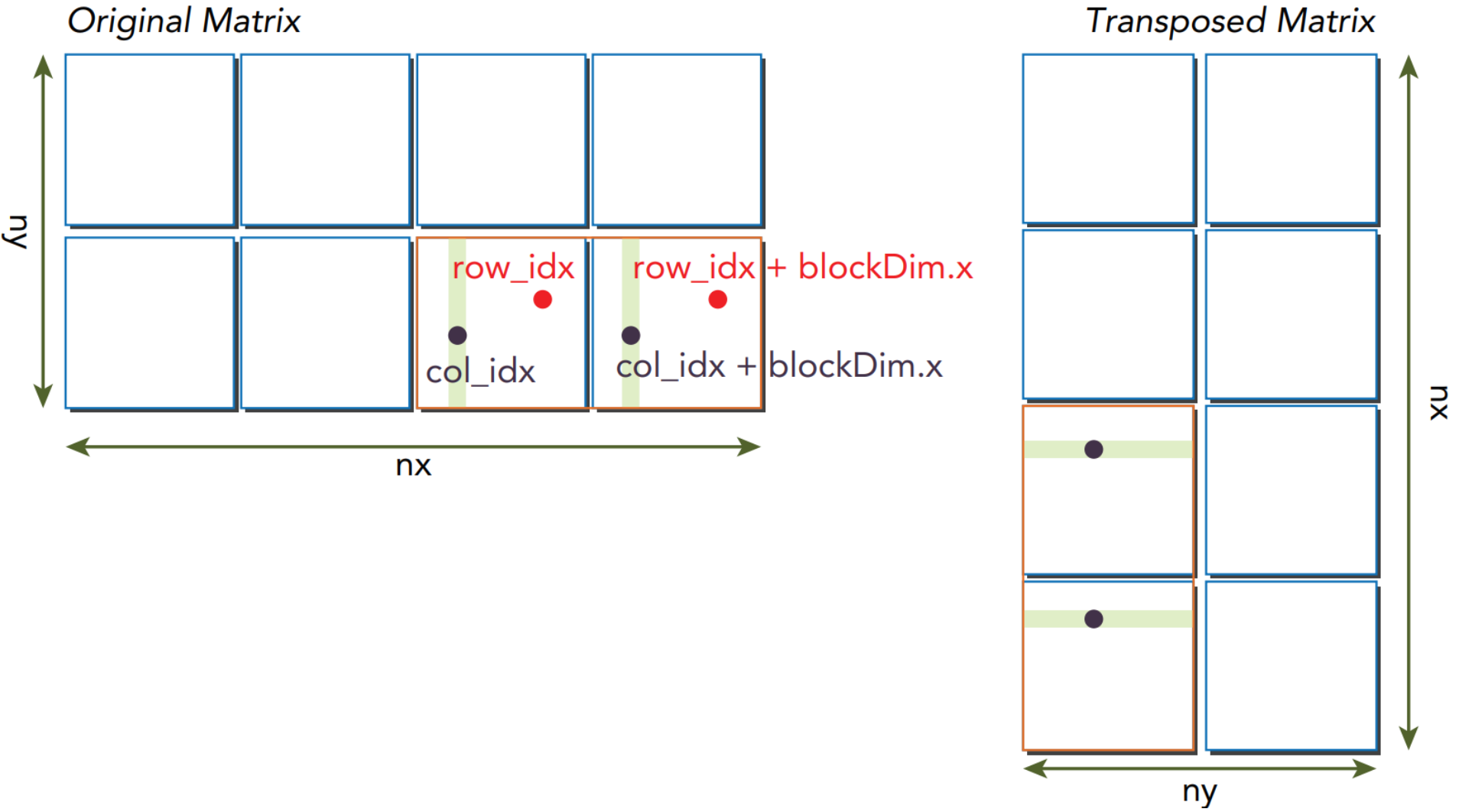
        // thread synchronization
        __syncthreads();

        // store two rows to global memory from two columns of shared memory
        unsigned int col_idx = icol*(blockDim.x*2+IPAD) + irow;
        out[to] = tile[col_idx];
        out[to+ny*BDIMX] = tile[col_idx+BDIMX];
    }
}
```



transposeRectangle.cu

Index Calculation



Performance of Transpose Kernels

KERNELS	TESLA M2090 (ECC OFF)		TESLA K40 (ECC OFF)	
	ELAPSED TIME (MS)	BANDWIDTH (GB/S)	ELAPSED TIME (MS)	BANDWIDTH (GB/S)
copyGmem (upper bound)	1.048	128.07	0.758	177.15
naiveGmem (lower bound)	3.611	37.19	1.947	68.98
transposeSmem	1.551	86.54	1.149	116.82
transposeSmemPad	1.416	94.79	1.102	121.83
transposeSmemUnrollPad	1.036	129.55	0.732	183.34
transposeSmemUnrollPadDyn	1.039	129.18	0.732	183.33

Dynamic shared memory allocation.
extern __shared__ float tile[];

- unrolling by two blocks provides a significant performance improvement
- more memory requests are simultaneously in-flight and read/write throughput improves

4.5 Exposing More Parallelism

Exposing More Parallelism

- One simple but effective optimization technique is to **adjust thread block dimensions**
  - to discover the best execution configuration.

KERNELS	BLOCK SIZE (32X32)		BLOCK SIZE (32X16)		BLOCK SIZE (16X16)	
	ELAPSED TIME (MS)	BANDWIDTH (GB/S)	ELAPSED TIME (MS)	BANDWIDTH (GB/S)	ELAPSED TIME (MS)	BANDWIDTH (GB/S)
`SmemUnrollPad`	0.779	172.21	0.732	183.34	0.723	185.66
`SmemUnrollPadDyn`	0.782	171.55	0.732	183.33	0.722	185.99

The block size 16×16 demonstrates the best performance because it exposes the most device parallelism with more concurrent thread blocks.

5. Constant Memory

Constant Memory （常量内存）

- **Constant memory** is a special-purpose memory used for data that is **read-only and accessed uniformly** by threads in a warp.

- While constant memory is **read-only from kernel** codes, it is both **readable and writable** from the **host**.

- Constant memory resides in **device DRAM** (like global memory) and has a **dedicated on-chip cache**.
  - There is 64 KB limit on the size of constant memory cache per SM.

Constant Memory

- Constant memory has a **different optimal access pattern**
  - It is best if all threads in a warp access the same location in constant memory.
  - Accesses to different addresses by threads within a warp are serialized.
  - The cost of a constant memory read scales linearly with the number of unique addresses read by threads within a warp.

- Constant variables must be declared in global scope with `__constant__`

- Constant memory variables exist for the **lifespan of the application**
  - accessible from all threads within a grid and from the host through runtime functions.

Constant Memory

- Values in constant memory must be **initialized** from **host code** with:

```
cudaError_t cudaMemcpyToSymbol(
      const void *symbol,
      const void * src,
      size_t count,
      size_t offset,
      cudaMemcpyKind kind)
```

symbol specifies the constant memory location on the device.

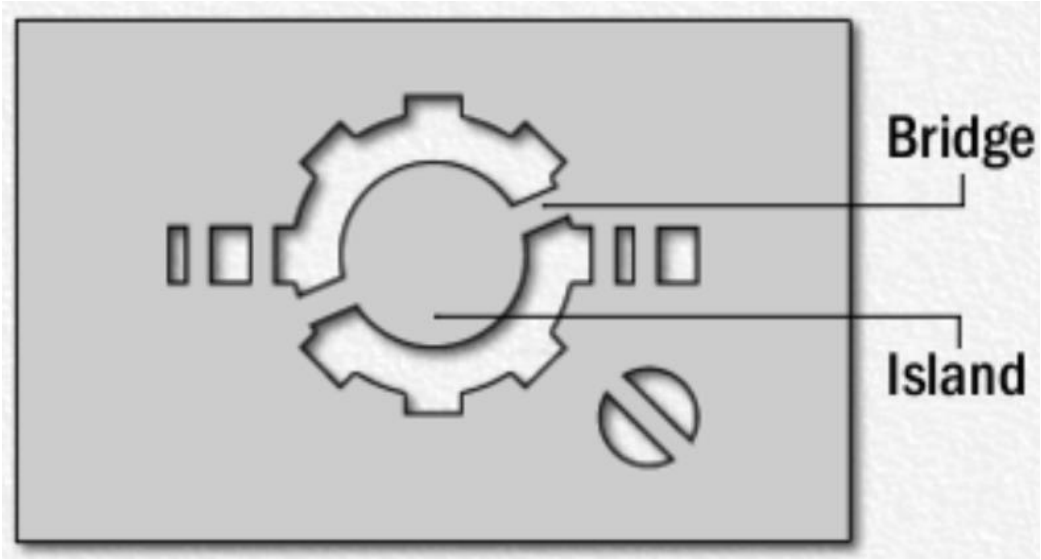
src points to data to be copied

kind specifies the direction of the transfer.
By default, kind is cudaMemcpyHostToDevice.

5.1 Implementing a 1D Stencil with Constant Memory

Stencil

- Stencil: 模板喷画



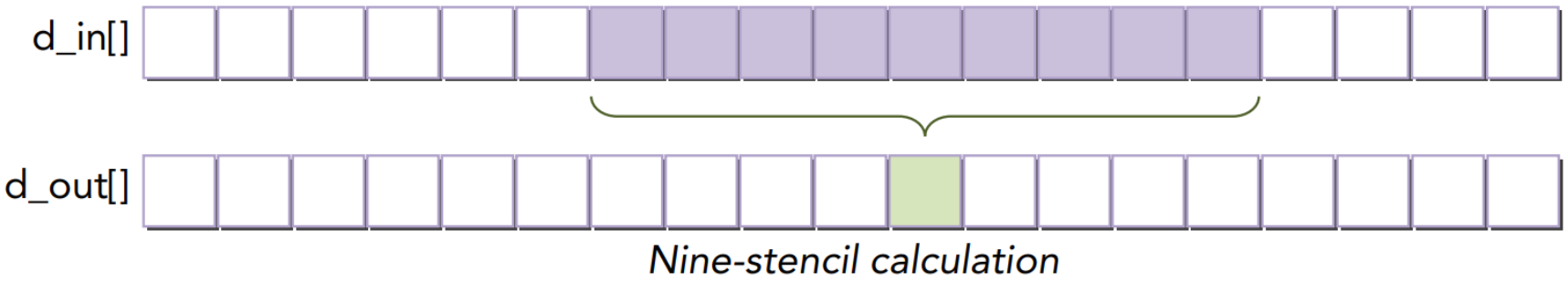




Stencil Computation (模板计算)

- A stencil computation applies **a function to a collection of geometric points** and updates the value of a **single point** with the output.
  - the basis for many algorithms that solve partial differential equations.

$\{x - 4h, x - 3h, x - 2h, x - h, x, x + h, x + 2h, x + 3h, x + 4h\}$



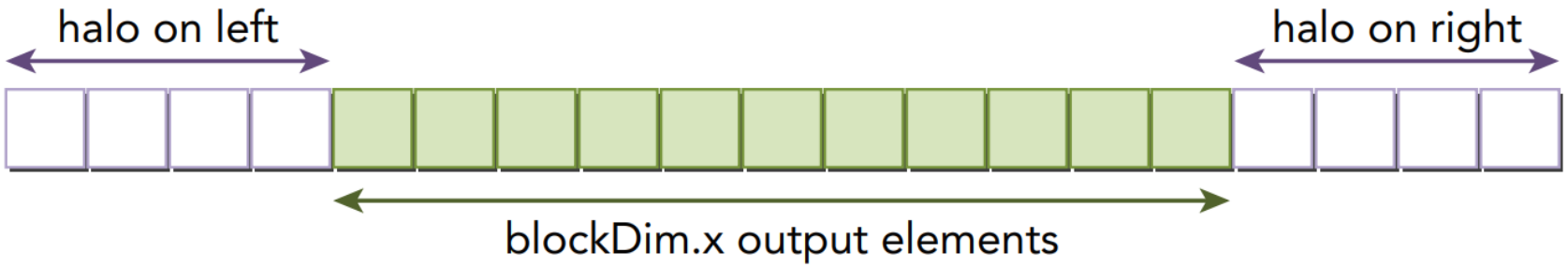
Nine-stencil calculation

$$f'(x) \approx c_0 (f(x + 4h) - f(x - 4h)) + c_1 (f(x + 3h) - f(x - 3h)) - c_2 (f(x + 2h) - f(x - 2h)) + c_3 (f(x + h) - f(x - h))$$

Stencil Computation

$$f'(x) \approx c_0(f(x+4h) - f(x-4h)) + c_1(f(x+3h) - f(x-3h)) - c_2(f(x+2h) - f(x-2h)) + c_3(f(x+h) - f(x-h))$$

coefficients *c0, c1, c2,* and *c3* are the same across all threads and ***excellent candidates*** for ***constant memory***




```
__shared__ float smem[BDIM + 2 * RADIUS];
```

use *shared memory* to cache data in order to reduce redundant accesses to global memory

```
__global__ void stencil_1d(float *in, float *out) {
   // shared memory
   __shared__ float smem[BDIM + 2*RADIUS];

   // index to global memory
   int idx = threadIdx.x + blockIdx.x * blockDim.x;

   // index to shared memory for stencil calculatioin
   int sidx = threadIdx.x + RADIUS;

   // Read data from global memory into shared memory
   smem[sidx] = in[idx];

   // read halo part to shared memory
   if (threadIdx.x < RADIUS) {
      smem[sidx - RADIUS] = in[idx - RADIUS];
      smem[sidx + BDIM] = in[idx + BDIM];
   }

   // Synchronize (ensure all the data is available)
   __syncthreads();

   // Apply the stencil
   float tmp = 0.0f;
   #pragma unroll
   for (int i = 1; i <= RADIUS; i++) {
        tmp += coef[i] * (smem[sidx+i] - smem[sidx-i]);
   }

   // Store the result
   out[idx] = tmp;
}
```



constantStencil.cu

reading data from global memory to shared memory

#pragma unroll is used as a hint to the CUDA compiler that this loop should be automatically unrolled

coef array is the constant memory array that holds the coefficients

Stencil Computation

- Declaring the **coef** array in constant memory is as simple as:

```
__constant__ float coef[RADIUS + 1];
```

- Initializing constant memory from the host side by **cudaMemcpyToSymbol**:

```
void setup_coef_constant(void) {

    const float h_coef[] =
        {a0, a1, a2, a3, a4};
    cudaMemcpyToSymbol(coef, h_coef,
        (RADIUS + 1) * sizeof(float));
}
```

5.2 Comparing with the Read-Only Cache

Read-Only Cache

- **Read-only cache** has **separate memory bandwidth** and can yield a performance benefit.

- Read-only cache is **better for scattered reads** than the L1 cache
  - should not be used when threads in a warp all read the same address.

- There is a total of **48 KB** of read-only cache per Kepler SM.
  - The granularity of the read-only cache is 32 bytes.

Read-Only Cache

- To access global memory through the read-only cache, there are two ways:
  - Using an intrinsic function **__ldg**
  - Qualifying pointers to global memory

```
__global__ void kernel(float* output, float* input) {
    ...
    output[idx] += __ldg(&input[idx]);
    ...
}
```

```
void kernel(float* output, const float* __restrict__ input) {
    ...
    output[idx] += input[idx];
}
```

Read-only cache vs. Constant cache

- Both constant cache and read-only cache are r**ead-only** from the **device**.
- Both have **limited** per-SM resources:
  - constant cache is 64 KB, and read-only cache is 48 KB.
- **Constant cache performs better** on **uniform reads**
  - where every thread in a warp accesses the same address.
- **Read-only cache** is better for **scattered reads**

Comparing with Read-Only Cache

allocate and initialize global memory to store the coefficients on the device

```
const float h_coef[] = {a0, a1, a2, a3, a4};
cudaMalloc((float**)&d_coef, (RADIUS + 1) * sizeof(float));
cudaMemcpy(d_coef, h_coef, (RADIUS + 1) * sizeof(float), cudaMemcpyHostToDevice);
```

```
__global__ void stencil_1d_read_only (float* in,
        float* out, const float *__restrict__ dcoef) {
  // shared memory
```

qualify pointers as const __restrict__ to indicate that they should be accessed through read-only cache



constantReadOnly.cu

```
Tesla K40c array size: 16777216 (grid, block) 524288,32
      3.4517ms  stencil_1d(float*, float*)
      3.6816ms  stencil_1d_read_only(float*, float*, float const *)
```

- Performance degrades if using read-only cache.
- The broadcast access pattern used for coef array is more optimized for constant memory than for the read-only

6. The Warp Shuffle Instruction

Warp Shuffle Instruction

- **shuffle instruction** allows a thread to **directly read** the **register** of another thread in **the same warp**.

- The shuffle instruction enables threads in a warp to **exchange data** with each other **directly**
  - rather than going through shared or global memory.

- The shuffle instruction has **lower latency than shared memory**
  - Not consume extra memory to perform a data exchange.

Lane

- A ***lane*** refers to a *single thread within a warp*.
  - Each lane in a warp is uniquely identified by a lane index in the range [0,31].
  - Each thread in a warp has a unique lane index
  - Multiple threads in the same thread block can have the same lane index.

- **No built-in** variable for lane index.

- In a 1D thread block, the lane index and warp index for a given thread can be calculated as:
  - `laneID = threadIdx.x % 32`
  - `warpID = threadIdx.x / 32`

6.1 Variants of the Warp Shuffle Instruction

Variants of Warp Shuffle Instruction

- **Two sets** of shuffle instructions:
  - one for integer variables, another for float variables.
- Each set has **four variants** of the shuffle instruction.
- To exchange integer variable within a warp, we can use:

```
int __shfl(int var, int srcLane, int width=warpSize);
```

returns the value var passed to __shfl by the thread in the same warp identified by srcLane.

This function makes it possible for each thread in a warp to directly fetch a value from a specific thread.

Variants of Warp Shuffle Instruction

int __shfl(int var, int srcLane, int width=warpSize);

- width can be set to any power-of-2 between 2 and 32
- setting width permits subdivision of a warp into segments containing width threads each
- a separate shuffle operation performed in each segment.

- For **width < 32**, the lane ID of a thread and its ID in a shuffle operation are not necessarily equal.
  - the shuffle ID of a thread in a 1D thread block is:

```
shuffleID = threadIdx.x % width;
```

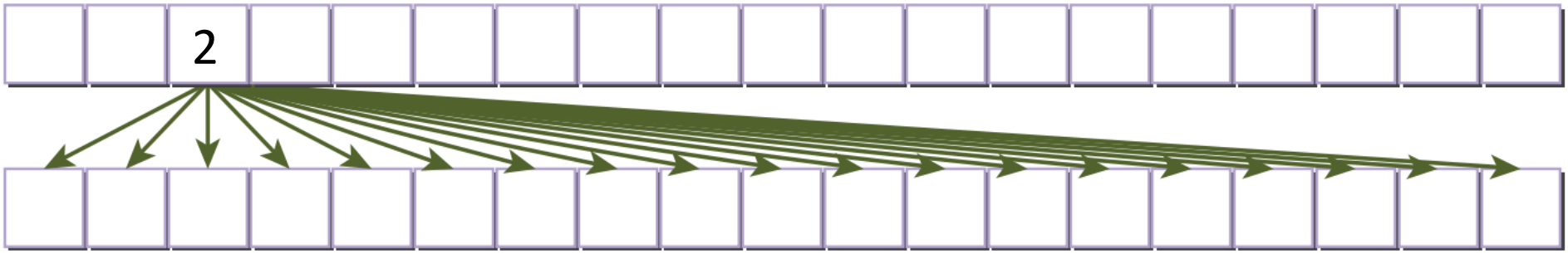
Examples

- if **`_shfl`** were called from every thread in a warp with :

```
int y = _shfl(x, 3, 16);
```

- threads 0-15 receive the value of x from thread 3
- threads 16-31 receive the value of x from thread 19
  - thread 19: thread with offset 3 in the top 16 threads of the warp

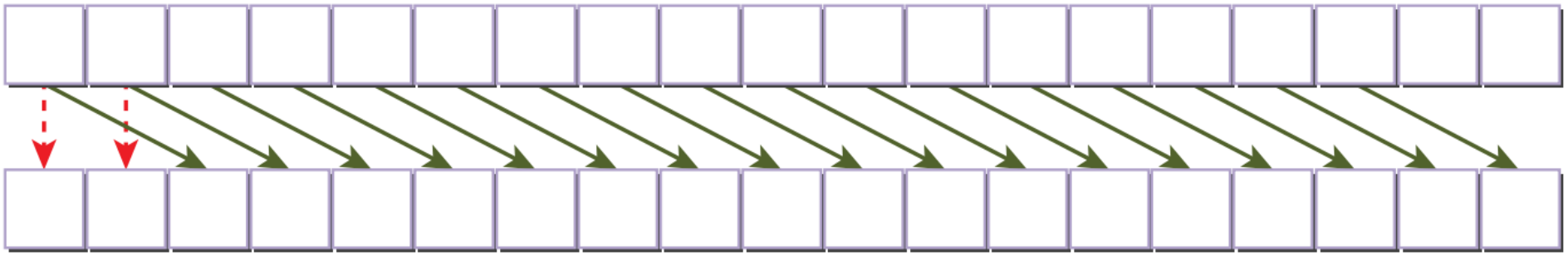
- **`_shfl(val,2):`** a broadcast from lane 2 to all threads in the warp



Variants of Warp Shuffle Instruction

- **`__shfl_up`** shifts **`var`** up the warp by delta lanes.

```
int __shfl_up(int var,
  unsigned int delta,
  int width=warpSize)
```

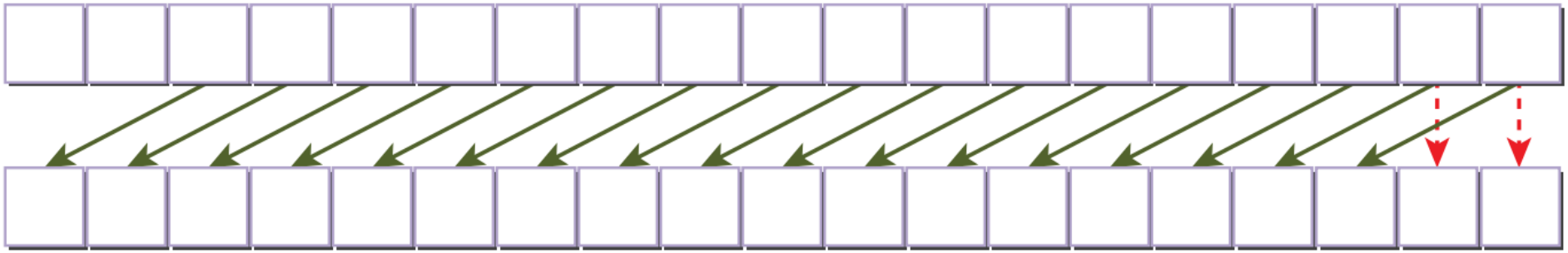


_shfl_up (val,2): Shift the value to the right two lanes.

Variants of Warp Shuffle Instruction

- **`__shfl_down`** shifts **`var`** down the warp by delta lanes.

```
int __shfl_down(int var,
    unsigned int delta,
    int width=warpSize)
```

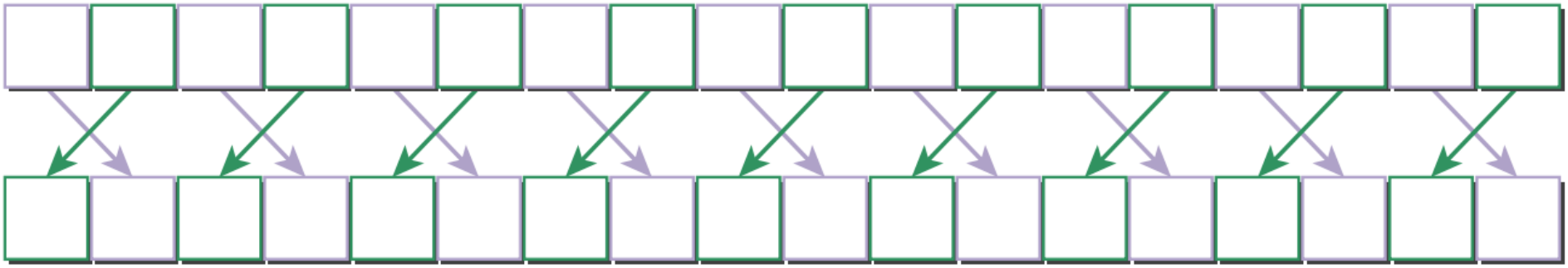


_shfl_down (val,2): Shift the value to the left two lanes.

Variants of Warp Shuffle Instruction

- **`__shfl_xor`** transfers data from a lane based on a bitwise XOR of the caller's own lane index:

```
int __shfl_xor(int var,
               int laneMask,
               int width=warpSize)
```



_shfl_xor (val,1): Implement a butterfly exchange.

The instruction calculates a source lane index by performing a bitwise XOR of the caller's lane index with laneMask.

6.2 Sharing Data within a Warp

Broadcast of a Value across a Warp

```
__global__ void test_shfl_broadcast(int *d_out, int *d_in, int const srcLane) {
    int value = d_in[threadIdx.x];
    value = __shfl(value, srcLane, BDIMX);
    d_out[threadIdx.x] = value;
}
```



simpleShfl.cu

- Each thread has one register variable **`value`**.
- The source lane is specified by the variable **`srcLane,`** which is equal across all threads.
- Each thread directly copies the value from the source thread.

- The kernel is invoked as follows.

```
test_shfl_broadcast<<<1, BDIMX>>>(d_outData, d_inData, 2);
```

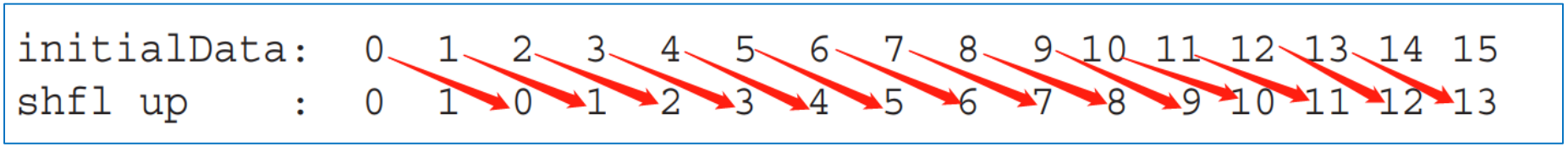
```
initialData:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
shfl bcast :  2  2  2  2  2  2  2  2  2  2  2  2  2  2  2  2
```

Shift Up within a Warp

```
__global__ void test_shfl_up(int *d_out, int *d_in, unsigned int const delta) {
   int value = d_in[threadIdx.x];
   value = __shfl_up(value, delta, BDIMX);
   d_out[threadIdx.x] = value;
}
```

- The kernel is invoked as follows.

  test_shfl_up<<<1, BDIMX>>>(d_outData, d_inData, 2);

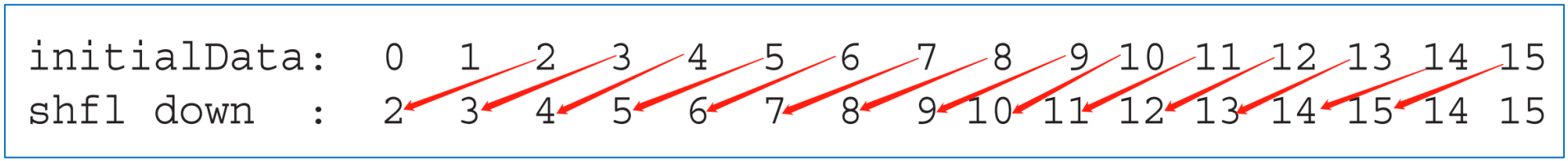


- The value of each thread is shifted to the right by two lanes.
- The values in the two leftmost lanes are unchanged

Shift Down within a Warp

```
__global__ void test_shfl_down(int *d_out, int *d_in, unsigned int const delta) {
   int value = d_in[threadIdx.x];
   value = __shfl_down(value, delta, BDIMX);
   d_out[threadIdx.x] = value;
}
```

- The kernel is invoked as follows.
  test_shfl_down<<<1, BDIMX>>>(d_outData, d_inData, 2);



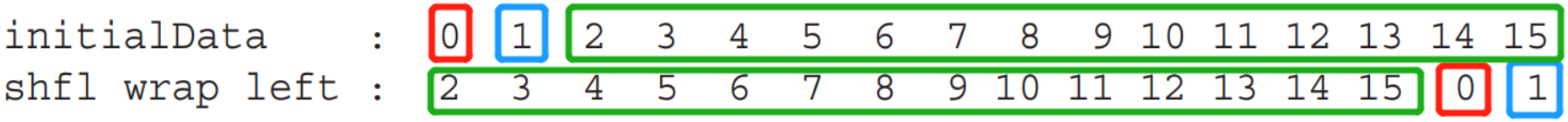
- The value of each thread is shifted to the left by two lanes.
- The values in the two rightmost lanes are unchanged

Shift within a warp with Wrap Around

```
__global__ void test_shfl_wrap(int *d_out, int *d_in, int const offset) {
    int value = d_in[threadIdx.x];
    value = __shfl(value, threadIdx.x + offset, BDIMX);
    d_out[threadIdx.x] = value;
}
```

- The kernel is invoked as follows.

  test_shfl_wrap<<<1,BDIMX>>>(d_outData, d_inData, 2);



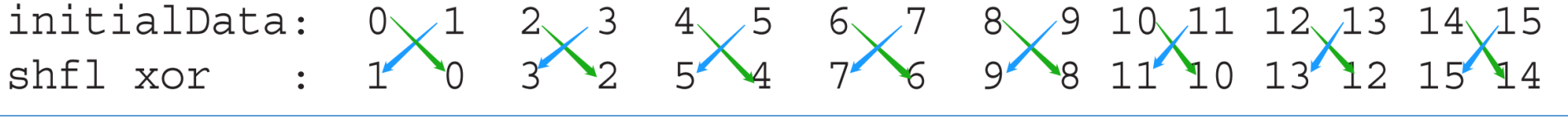
- This kernel implements a shift-to-left wrap-around operation
- The values for the two rightmost lanes are changed here

Butterfly Exchange across the Warp

```
__global__ void test_shfl_xor(int *d_out, int *d_in, int const mask) {
    int value = d_in[threadIdx.x];
    value = __shfl_xor (value, mask, BDIMX);
    d_out[threadIdx.x] = value;
}
```

- The kernel is invoked as follows.

  test_shfl_xor<<<1, BDIMX>>>(d_outData, d_inData, 1);



- This kernel implements a butterfly addressing pattern between two threads
- Invoking the kernel with mask as 1 will result in adjacent threads exchanging their values.

Exchange Values of an Array across a Warp

```
__global__ void test_shfl_xor_array(int *d_out, int *d_in, int const mask) {
   int idx = threadIdx.x * SEGM;
   int value[SEGM];

   for (int i = 0; i < SEGM; i++) value[i] = d_in[idx + i];

   value[0] = __shfl_xor (value[0], mask, BDIMX);
   value[1] = __shfl_xor (value[1], mask, BDIMX);
   value[2] = __shfl_xor (value[2], mask, BDIMX);
   value[3] = __shfl_xor (value[3], mask, BDIMX);

   for (int i = 0;i < SEGM; i++) d_out[idx + i] = value[i];
}
```

#define SEGM 4

```
test_shfl_xor_int4<<<1, BDIMX / SEGM>>>(d_outData, d_inData, 1);
```

threadIdx.x	0	1	2	3
initialData:	0 1 2 3	4 5 6 7	8 9 10 11	12 13 14 15
shfl array :	4 5 6 7	0 1 2 3	12 13 14 15	8 9 10 11

Exchange Values Using Array Indices Across a Warp

- swap elements between two threads at different offsets

```
__inline__ __device__
void swap(int *value, int laneIdx, int mask, int firstIdx, int secondIdx) {
    bool pred = ((laneIdx / mask + 1) == 1);
    if (pred) {
        int tmp = value[firstIdx];
        value[firstIdx] = value[secondIdx];
        value[secondIdx]= tmp;
    }

    value[secondIdx] = __shfl_xor(value[secondIdx], mask, BDIMX);

    if (pred) {
        int tmp = value[firstIdx];
        value[firstIdx] = value[secondIdx];
        value[secondIdx]= tmp;
    }
}
```

pred is used to identify the first calling thread in a pair of threads that are exchanging data.

The first calling thread starts by swapping the elements at firstIdx and secondIdx, but only in its local array.

A butterfly exchange is then performed on the location secondIdx between two threads.

the first thread swaps the element received in secondIdx back to firstIdx.

Exchange Values Using Array Indices Across a Warp

- The following kernel swaps two elements at different offsets between two threads based on the preceding swap function.

```
__global__
void test_shfl_swap(int *d_out, int *d_in, int const mask, int firstIdx,
                    int secondIdx) {
   int idx = threadIdx.x * SEGM;
   int value[SEGM];

   for (int i = 0; i < SEGM; i++) value[i] = d_in[idx + i];

   swap(value, threadIdx.x, mask, firstIdx, secondIdx);

   for (int i = 0; i < SEGM; i++) d_out[idx + i] = value[i];
}

test_shfl_swap<<<1, block / SEGM >>>(d_outData, d_inData, 1, 0, 3);
```



Exchange Values Using Array Indices Across a Warp

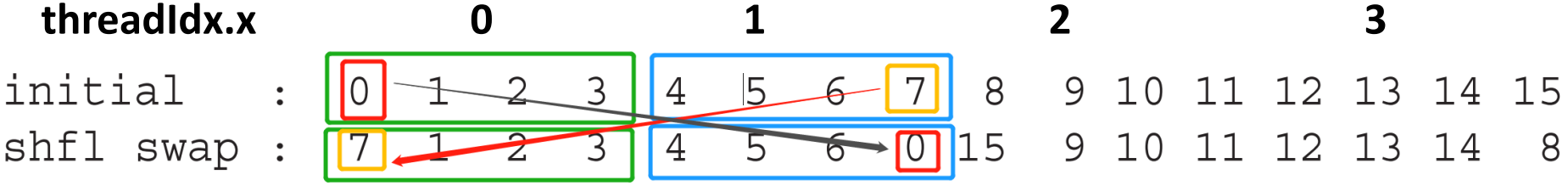
```
test_shfl_swap<<<1, block / SEGM >>>(d_outData, d_inData, 1, 0, 3);
```

mask

1st index

2nd index

- for each pair of threads, the 1st element in the array of the 1st calling thread is swapped with the 4th element in the array of the 2nd calling thread



Thread 0: 0123 → 3120 → 3127 → 7123

Thread 1: 4567 → 4567 → 4560

6.3 Parallel Reduction Using the Warp Shuffle Instruction

Parallel Reduction Using the Warp Shuffle Instruction

- Learn how to solve the reduction problem using **warp shuffle** instructions.

- It consists of **three levels of reduction**:
  - Warp-level reduction
  - Block-level reduction
  - Grid-level reduction

Warp-level Reduction

- For warp-level reduction, each warp performs its own reduction.
  - Instead of using shared memory, each thread uses a register to store one data element, reading from global memory

```
__inline__ __device__ int warpReduce(int mySum) {
  mySum += __shfl_xor(mySum, 16);
  mySum += __shfl_xor(mySum, 8);
  mySum += __shfl_xor(mySum, 4);
  mySum += __shfl_xor(mySum, 2);
  mySum += __shfl_xor(mySum, 1);
  return mySum;
}
```

- Each warp sum is saved to shared memory based on thread index and warp size

```
int laneIdx = threadIdx.x % warpSize;
int warpIdx = threadIdx.x / warpSize;
mySum = warpReduce(mySum);
if (laneIdx == 0) smem[warpIdx] = mySum;
```

Block-level and Grid-level Reduction

- **Block-level Reduction**
  - The block is synchronized and the same warp reduction is used to add together each warp's sum.
  - The final output produced by the block is then saved to global memory by the first thread in the block

```
__syncthreads();
mySum = (threadIdx.x < SMEMDIM) ? smem[laneIdx] : 0;
if (warpIdx == 0) mySum = warpReduce(mySum);
if (threadIdx.x == 0) g_odata[blockIdx.x] = mySum;
```

- **Grid-level Reduction**
  - **`g_odata`** is copied back to the host where the final reduction is performed.

```
    // save warp sum to shared memory
    if (laneIdx==0) smem[warpIdx] = mySum;

    // block synchronization
    __syncthreads();

    // last warp reduce
    mySum = (threadIdx.x < SMEMDIM) ? smem[laneIdx]:0;
    if (warpIdx==0) mySum = warpReduce (mySum);

    // write result for this block to global mem
    if (threadIdx.x == 0) g_odata[blockIdx.x] = mySum;
}
```



reduceIntegerShfl.cu

Time(%)	Time	Calls	Avg	Min	Max	Name
24.95%	4.0000us	1	4.0000us	4.0000us	4.0000us	reduceSmem()
17.76%	2.8480us	1	2.8480us	2.8480us	2.8480us	reduceShfl()

1.42 times speedup from using the shuffle instruction

Summary

Summary

- Shared memory can be declared as a **1D or 2D array**, providing a simple logical view to ease programming.
  - shared memory is accessed through 32 banks.
  - Avoiding bank conflicts is important factor during optimization.

- There are **two main reasons** to use shared memory in a kernel.
  - One is to cache data on-chip and reduce the amount of global memory traffic.
  - The other is to transform how data is arranged in shared memory to avoid non-coalesced global memory accesses.

Summary

- **Constant memory** is optimized for **read-only data** that is broadcast to many threads at a time.
- The **read-only texture cache** is more optimized for **scattered reads** of data.
- The **shuffle** instruction is a **warp-level intrinsic function**
  - enable threads in a warp to share data with each other quickly and directly.
  - shuffle instruction has lower latency than shared memory
  - Not require allocating additional resources.